

Fast AVX-512 Implementation of the Optimal Ate Pairing on BLS12-381

Hao Cheng^{1,2,3}, Georgios Fotiadis^{4,5}, Johann Großschädl⁵ and Daniel Page⁶

¹ School of Cyber Science and Technology, Shandong University, Qingdao, China

² State Key Laboratory of Cryptography and Digital Economy Security, Shandong University, Qingdao, China

³ Key Laboratory of Cryptologic Technology and Information Security, Ministry of Education, Shandong University, Qingdao, China

hao.cheng@sdu.edu.cn

⁴ =nil; Foundation, Limassol, Cyprus

gfotiadis.crypto@gmail.com

⁵ DCS and SnT, University of Luxembourg, Esch-sur-Alzette, Luxembourg

johann.groszschaedl@uni.lu

⁶ School of Computer Science, University of Bristol, Bristol, UK

daniel.page@bristol.ac.uk

Abstract. Non-degenerate bilinear maps on elliptic curves, commonly referred to as *pairings*, have many applications including short signature schemes, zero-knowledge proofs and remote attestation protocols. Computing a state-of-the-art pairing at the 128-bit security level, such as the optimal ate pairing over the curve BLS12-381, is very costly due to the high complexity of some of its sub-operations: most notable are the Miller loop and final exponentiation. In the past ten years, a few optimized pairing implementations have been introduced in the literature, but none of those took advantage of the vector (SIMD) extensions of state-of-the-art Intel and AMD CPUs, especially AVX-512; this is surprising, because doing so offers the potential to reach significant speed-ups. Consequently, the questions of 1) how computation of the optimal ate pairing can be effectively vectorized, and 2) what execution time such a vectorized implementation can achieve are still open. This paper addresses said questions by introducing a carefully-optimized AVX-512 implementation of the optimal ate pairing on BLS12-381. A central feature of the implementation is the use of 8-way *Integer Fused Multiply-Add (IFMA)* instructions, which are capable to execute eight 52×52 -bit multiplications in a SIMD-parallel fashion. We introduce new vectorization strategies and describe optimizations of existing ones to speed up arithmetic operations in the extension fields $\mathbb{F}_{p^4}$, $\mathbb{F}_{p^6}$, and $\mathbb{F}_{p^{12}}$ as well as certain higher-level functions. Furthermore, we discuss some parallelization bottlenecks and how they impact execution time. We benchmarked our pairing software, which we call `avxb1s`, on an Intel Core i3-1005G1 (“Ice Lake”) CPU and found that it needs 1,265,314 clock cycles (resp. 1,195,236 clock cycles) for the full pairing, with the Granger-Scott cyclotomic squaring (resp. compressed cyclotomic squaring) being used in the final exponentiation. For comparison, the non-vectorized (i.e., scalar) x64 assembly implementation from the widely-used `blst` library has an execution time of 2,351,615 cycles, which is 1.86 times (resp. 1.97 times) slower. `avxb1s` also outperforms Longa’s implementation (CHES 2023) by almost the same factor. The practical importance of these results is amplified by Intel’s recent announcement to support AVX10, which includes IFMA instructions, in all future CPUs.

Keywords: Pairing-based cryptography, Optimal ate pairing, SIMD, AVX-512

1 Introduction

A *pairing* is generally defined as a function e that maps a pair $(P, Q) \in \mathbb{G}_1 \times \mathbb{G}_2$ to an element $e(P, Q) \in \mathbb{G}_T$, where $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ are groups of prime order r . The map e is usually bilinear, non-degenerate, and (efficiently) computable [BSS05]. Starting 24 years ago with pioneering works such as [BF01, BLS01, Jou04], which introduced pairings in cryptography, they have been used as a fundamental building block in digital signature schemes [BB04, PS16, TZ23], polynomial commitments [KZG10], zero-knowledge proofs (e.g., zkSNARKs [Gro16]), anonymous credentials [CL04], as well as (remote) attestation protocols [BCL08, CMS08]. In cryptographic applications, pairings over ordinary elliptic curves are now considered as the standard choice [BSS05]. The construction of suitable *pairing-friendly* elliptic curves has been widely studied, with many potential candidates of such curves identified in [FST10]. However, the improved Special Tower Number Field Sieve (STNFS) attacks [KB16] then had a massive impact by significantly reducing the estimated security level offered by several curves. In response, various subsequent papers provided new recommendations and estimates on the claimed security level, taking into account the improved STNFS attacks: see, e.g., [BD19, FK19, Gui20, MAF20, AFG24].

Optimal ate pairing on BLS12-381. The BLS12-381 curve [Bow17] is a prominent and widely-used example of such a new elliptic curve for pairing-based cryptosystems. As its name indicates, BLS12-381 belongs to the Barreto-Lynn-Scott (BLS) family of pairing-friendly curves [BLS02], has an embedding degree of 12, and is defined over a prime field $\mathbb{F}_p$ with p of size 381 bits. The *Optimal Ate Pairing (OAP)* [Ver10] on this elliptic curve is known to be the most efficient pairing at the 128-bit security level. To be more precise, this pairing is estimated to offer 126 bits of security with respect to the improved STNFS attacks [GS21]. The BLS12-381 curve has already been integrated in well-known cryptographic libraries, and adopted by organizations like Ethereum and Zcash, which operate in the blockchain and cryptocurrency space. For example, the OAP over BLS12-381 is used in the Zcash project for constructing zkSNARK proof systems [Bow17]. It is also the fundamental building block of the Boneh-Lynn-Shacham (BLS) short signature scheme [BLS01], which is currently deployed in the Ethereum 2.0 project [Edg23].

From an arithmetic point of view, the execution time of the OAP over BLS12-381 is significantly impacted by the time needed for certain operations (especially multiplication and squaring) in the underlying prime field $\mathbb{F}_p$ and extensions of it. When implemented in software, these operations are performed on multiple-precision integers and have to be optimized for the (micro-)architectural features of the target CPU. Examples of these features are the width w of the multiplier, the latency of a $w \times w$ -bit multiplication, and how many $w \times w$ -bit multiplications can be performed in parallel via instruction-level parallelism (i.e., superscalar execution) or SIMD parallelism (i.e., vector processing).

Intel Advanced Vector eXtensions (AVX) family. The history of SIMD processing in Intel CPUs started in the late 1990s with the introduction of the MultiMedia eXtension (MMX), soon followed by different versions of Streaming SIMD Extensions (SSE). This evolution from MMX to SSE did not only increase the number of logical SIMD registers from 8 to 16 (on x86-64) and their width from 64 bits to 128 bits, but also improved the orthogonality of instructions (MMX supported only 32-bit elements, which was relaxed to 8, 16, 32, or 64-bit elements in SSE2). In 2011, Intel introduced the first version of the *Advanced Vector eXtensions (AVX)*, focusing mainly on floating-point operations. Unlike MMX and SSE, AVX instructions have a three-register format with operands/results in 16 registers, each 256 bits wide. AVX2, released in 2013, extended these SIMD features to include integer operations. The next step in the evolution of AVX took place in 2016 when Intel launched AVX-512, which further increased the number of SIMD registers to

32 and their size to 512 bits. In addition, Intel developed a couple of optional extensions to AVX-512, such as AVX-512IFMA, which was introduced with the Cannon Lake micro-architecture. AVX-512IFMA was specifically developed to accelerate multiple-precision integer arithmetic through two *Integer Fused Multiply-Add (IFMA)* instructions named `vpmmadd52luq` and `vpmmadd52huq` [Int25b]. These instructions are extremely powerful and combine an 8-way 52×52 -bit multiplication with an addition of either the lower or the higher half of the eight products to eight 64-bit elements of a 512-bit vector.

Many software speed records on modern Intel processors presented in the literature were achieved thanks to the massive parallel processing capabilities of AVX instructions: see, e.g., [Cho15, FL15, FLD19, HEY20, CGT⁺20, NS21, ZHZ⁺24] for “classical” elliptic curve cryptosystems and, e.g., [KG19, CFG⁺21, CFGR22] for isogeny-based schemes. In particular, lib25519 [BN24], which provides the currently fastest implementations of the widely-used X25519 and Ed25519 algorithms, is carefully optimized for different versions of AVX. Until now, AVX-512IFMA has received rather little attention, especially in the context of pairing-based cryptography, because it is only supported by a relatively small fraction of Intel’s current CPUs. However, this will change in the coming years as Intel announced in 2023 the introduction of AVX10 [Int25a], which will unify support for AVX-512 capabilities, including IFMA instructions, across both P-cores and E-cores in future CPUs. Earlier revisions of [Int25a] from 2023 indicated that only a “converged” version of AVX10 with 256-bit vector registers will be supported on E-cores, but revision 3 from March 2025 makes it clear that both P and E cores will have full 512-bit vector registers and be able to execute 512-bit AVX instructions¹. Besides Intel CPUs, AVX-512 with IFMA is also available on all AMD CPUs based on the Zen5 micro-architecture, which actually features a 512-bit datapath² to support AVX-512. Therefore, IFMA instructions will soon be omnipresent on both Intel and AMD CPUs.

Challenges to the vectorization of pairings. SIMD instructions for multiple-precision integer multiplication, such as IFMA, offer many options to speed up the field-arithmetic performed by pairings. For example, when implementing multiplication in $\mathbb{F}_p$, one can use all eight 52×52 -bit multipliers of a AVX-512IFMA vector engine to perform one single instance of $\mathbb{F}_p$ -multiplication (such a (1×8) -way approach is referred to as “horizontal vectorization” in [BKD⁺12]). At the opposite end of the vectorization design space is an (8×1) -way implementation executing eight instances of $\mathbb{F}_p$ -multiplication at once, where each instance occupies a 64-bit element of a 512-bit AVX register and uses only a single 52×52 -bit multiplier (“vertical vectorization”). This design space has more dimensions for multiplication in extensions of $\mathbb{F}_p$, such as $\mathbb{F}_{p^2}$ or $\mathbb{F}_{p^4}$. For example, a multiplication in $\mathbb{F}_{p^4}$ can be composed of two parallel multiplications in $\mathbb{F}_{p^2}$, each of which can in turn be performed by two parallel multiplications in $\mathbb{F}_p$, and each of these $\mathbb{F}_p$ -multiplications can use two 64-bit elements of a 512-bit vector. Exploring this multi-dimensional design space is anything but easy since a multitude of (micro-)architectural aspects have to be taken into account when assessing different vectorization strategies. These aspects span instruction scheduling (i.e., reducing the impact of sequential dependencies or congestion of the execution ports), register allocation (i.e., minimizing register spilling), and memory accesses (i.e., making effective use of data and instruction caches).

These challenges are more pronounced for CPUs with a 256-bit vector unit, such as AVX2 and the 256-bit SVE extension of the ARMv8-A architecture, which can perform four 32×32 -bit multiplications in parallel, each producing a “full” 64-bit result. When using such a vector engine, a 381-bit integer has to be represented by an array of limbs of a length of at most 28 bits, which means each operand of a (4×1) -way multiplication occupies $\lceil 381/28 \rceil = 14$ ymm registers. In other words, an operand consumes 87.5% of the

¹See <https://www.phoronix.com/news/Intel-AVX10-Drops-256-Bit>.

²See https://www.numberworld.org/blogs/2024_8_7_zen5_avx512_takedown.

ymm register space, and this makes the register allocation highly challenging. A product-scanning version of a (4×1) -way multiplication executes $14^2 = 196$ `vpmuludq` instructions and roughly the same number of `vpaddq` instructions. However, when AVX-512IFMA is available, a 381-bit integer can be represented via eight 48-bit limbs, and each operand of an (8×1) -way multiplication occupies eight `zmm` registers. Therefore, both operands and the 762-bit product fit into the 32 `zmm` registers. A product-scanning version of an (8×1) -way multiplication consists of 64 `vpadd52luq` and 64 `vpadd52huq` instructions (i.e., 128 vector-multiply instructions altogether). No dedicated vector-add instructions are necessary, since additions of the lower/upper halves of the eight 104-bit products are part of IFMA, which reduces port congestion and simplifies instruction scheduling.

Related work. Grabher, Großschädl, and Page [GGP08] present SIMD implementations of the ate pairing on a Barreto-Naehrig (BN) curve [BN05] defined over a 256-bit prime field $\mathbb{F}_p$. These implementations utilize the 2-way parallelism provided by Intel’s SSE2/3 extension to perform simultaneously 1) two digit-wise operations to accelerate a single $\mathbb{F}_p$ -operation, 2) two $\mathbb{F}_p$ operations to speed up a single $\mathbb{F}_{p^2}$ operation, or 3) two pairing computations to improve throughput. Aranha, López, and Hankerson [ALH10] describe a vectorized implementation of the eta pairing over binary supersingular curves at the 128-bit security level, taking advantage of SSE2 instructions to speed up the polynomial arithmetic, i.e., the operations in $\mathbb{F}_{2^n}$. Their implementation uses an elliptic curve with embedding degree 4 defined over $\mathbb{F}_{2^{1223}}$ as a case study. Different vectorization strategies for SSE2 and AVX2 to optimize the eta pairing on binary and ternary elliptic curves are also studied in [BKD⁺12]. In summary, it is surprising that there are only three papers on vectorized implementation of pairings, and all of them were published over ten years ago, targeting Intel CPUs that are significantly behind the state-of-the-art.

Contributions. The above summary of relevant previous work shows that the question of how to leverage powerful SIMD extensions, such as AVX-512IFMA, for pairing-based cryptography has not been researched so far. With the present paper, we aim to fill this gap and describe, to the best of our knowledge, the first AVX-512IFMA implementation of a pairing computation, whereby we target the OAP on the BLS12-381 curve. We used version 0.3.13 of `blst`, a well-known BLS signature library³ focused on performance and security, as starting point for our work. `blst` is written in C and Assembly for x64 and ARM64, but is a scalar implementation and does not take advantage of vector engines that are available for these platforms. We identified six performance-critical operations in `blst` (four $\mathbb{F}_{p^{12}}$ operations and two line operations) and propose suitable vectorization strategies for them, which span several layers including $\mathbb{F}_p$, $\mathbb{F}_{p^2}$, $\mathbb{F}_{p^6}$ or $\mathbb{F}_{p^4}$, and $\mathbb{F}_{p^{12}}$. In addition, we analyze different low-level implementation techniques (e.g., multiplication algorithms: schoolbook vs. Karatsuba, and reduction strategies: using vs. not using lazy reduction) and different degrees of parallelism: 2-way, 4-way, and a hybrid variant. All these vectorized operations are integrated in `avxbls`, a latency-optimized implementation of the OAP on BLS12-381 using AVX-512IFMA vector instructions. Furthermore, we extended `blst` with support for compressed cyclotomic squaring [AKL⁺11], in addition to its native Granger-Scott cyclotomic squaring [GS10]. Both `blst` and `avxbls` meet all usual requirements for “constant-timeness” and are resistant against timing attacks. We benchmarked the two libraries on an Intel Core i3-1005G1 CPU and found `avxbls` to be roughly 1.86 times (resp., 1.97 times) faster for the full pairing computation than the original x64 assembly version of `blst`, when `avxbls` uses the Granger-Scott cyclotomic squaring [GS10] (resp., the compressed cyclotomic squaring introduced in [AKL⁺11]) in the final exponentiation.

³For example, Ethereum uses `blst` for their implementation of polynomial commitments and various other projects. The source code of `blst` is available at <https://github.com/supranational/blst>.

Our `avxb1s` library also outperforms Longa’s (scalar) OAP implementation over the BLS12-381 curve described in [Lon23] by a factor of 1.85. Even though the optimization techniques applied by Longa are well documented, a direct comparison with `avxb1s` is intricate since the (micro-)architectural features and constraints of x64 differ much from that of AVX-512IFMA. For example, the latter offers 32 512-bit registers, while x64 has only 16 registers of 64 bits. Longa’s implementation aims to minimize the execution time by optimizing the register allocation and making ideal use of the BMI2/ADX extensions (e.g., `mulx`). The register pressure is much lower with AVX-512, and there is no need to optimize register allocation. Instead, the goals of `avxb1s` are the best-possible utilization of the eight 52×52 -bit multipliers and the reduction of vectorization overheads.

The source code of our `avxb1s` software is publicly available⁴ under an open source license.

Organization. Section 2 provides background information covering topics such as the OAP on the BLS12-381 curve and the IFMA instructions. It also specifies the notation used in this paper. In Section 3, we explain the vectorization strategies for six essential operations of the target pairing, followed by details on the integration of our AVX-512 functions into `blst`. We study the performance of `avxb1s`, `blst`, and Longa’s software in Section 4. Finally, in Section 5, we draw conclusions and discuss future work.

2 Background

2.1 OAP over the BLS12-381 curve

BLS12-381. BLS12-381 is an elliptic curve discovered by Bowe [Bow17] that belongs to the BLS family [BLS02] of “pairing-friendly” curves with embedding degree 12, in this paper abbreviated by BLS12. The BLS12 family supports a parametric description since the curve parameters can be represented via polynomials $t(x), r(x), p(x) \in \mathbb{Q}[x]$, where $t(x) = x + 1$, $r(x) = x^4 - x^2 + 1$, and $p(x) = (x - 1)^2(x^4 - x^2 + 1)/3 + x$ correspond to the trace of Frobenius, the prime factor of the curve-order, and the underlying base field $\mathbb{F}_p$, respectively. BLS12 curves do not only share the same embedding degree, but also have the Complex Multiplication (CM) discriminant 3 and j -invariant 0 in common. In addition, they are described by the equation $E/\mathbb{F}_p : y^2 = x^3 + b$, where $b \neq 0$, and have order $\#E(\mathbb{F}_p) = h(x) \cdot r(x)$ for some polynomial *co-factor* $h(x) \in \mathbb{Q}[x]$. A BLS12 curve admits degree-6 twists, which facilitates the efficient implementation of the OAP.

Members of the BLS12 family can be generated by evaluating the three polynomials $t(x)$, $r(x)$, and $p(x)$ at some *seed* $z \in \mathbb{Z}$ satisfying $z \equiv 1 \pmod{3}$. The BLS12-381 curve is obtained by evaluating these polynomials at the seed $z = -0\text{xd}201000000010000$; this yields a prime $p = p(z)$ of length 381 bits and a prime $r = r(z)$ of length 255 bits. The parameter b of the curve is 4 and the seed z has a low Hamming weight of only 6, which enables further performance gains since it reduces the number of addition steps in the Miller loop of the OAP. Furthermore, the low Hamming weight of the seed improves the performance of the final exponentiation by reducing the cost of the five exponentiations by z it has to perform. Based on the evaluation in [Gui20], the OAP on BLS12-381 has a security level of 126 bits when taking the improved STNFS attacks into account.

OAP. The OAP was introduced by Vercauteren in [Ver10] as an optimization of the ate pairing [HSV06] that reduces the length of the Miller loop. To maximize efficiency, the formula for computing the OAP has to be tailored to the characteristics of the curve; we refer to [Ver10] for the generic definition of the OAP and concentrate here primarily on

⁴See <https://github.com/hchengv/avxb1s>.

Algorithm 1: Miller loop.

Input: $P \in \mathbb{G}_1$, $Q \in \mathbb{G}_2$ and $|z| = (b_n, b_{n-1}, \dots, b_1, b_0)$ with $\lfloor \log_2(|z|) \rfloor = n$.
Output: Miller function $f = f_{z,Q}(P)$.

```

1  $f \leftarrow 1, T \leftarrow Q$ 
2 for  $i \leftarrow \lfloor \log_2(|z|) \rfloor - 1$  down to 0 do
3    $\ell \leftarrow \ell_{T,T}(P), T \leftarrow [2]T$  /* Line computation/evaluation and point doubling */
4    $v \leftarrow v_{[2]T}(P)$  /* Vertical line */
5    $f \leftarrow f^2 \cdot \ell / v$  /* Update Miller function */
6   if  $b_i = 1$  then
7      $\ell \leftarrow \ell_{T,Q}(P), T \leftarrow T + Q$  /* Line computation/evaluation and point addition */
8      $v \leftarrow v_{T+Q}(P)$  /* Vertical line */
9      $f \leftarrow f \cdot \ell / v$  /* Update Miller function */
10 if  $z < 0$  then
11    $f \leftarrow f^{-1}$ 
12 return  $f$ 

```

BLS12 curves. The computation of the OAP involves two main steps: the Miller loop to compute the Miller function f and the final exponentiation. Targeting specifically BLS12 curves, let $\mathbb{G}_2 = E(\mathbb{F}_{p^{12}})[r] \cap \ker(\pi_p - [p])$ and $\mathbb{G}_1 = E(\mathbb{F}_p)[r] \cap \ker(\pi_p - [1])$, where $\pi_p : E \rightarrow E$ is the Frobenius endomorphism with $(x, y) \mapsto (x^p, y^p)$. Further, let $\mathbb{G}_T$ be the group containing the r^{th} -roots of unity in $\mathbb{F}_{p^{12}}$, and we write $\mathbb{G}_T = \mu_r \subset \mathbb{F}_{p^{12}}^*$. In this setting, the OAP on the BLS12-381 curve is defined as the non-degenerate, bilinear map $e : \mathbb{G}_2 \times \mathbb{G}_1 \rightarrow \mathbb{G}_T$ such that for any $Q \in \mathbb{G}_2$ and $P \in \mathbb{G}_1$ we have

$$(Q, P) \mapsto f_{z,Q}(P)^{\frac{p^{12}-1}{r}}$$

where $z \in \mathbb{Z}$ is the seed used to derive BLS12-381.

The Miller function is computed using the Miller loop [Mil04], which is an iterative procedure, with $\lfloor \log_2(z) \rfloor = 63$ iterations composed of a doubling step and an addition step. Each iteration executes the doubling step, which includes the computation of the line function $\ell_{T,T}(P)$, i.e., the tangent line passing through T and evaluated at P , the computation of the point $[2]T$, and an update on the Miller function f . An addition step is executed for every “1” bit in the binary expansion of z . It computes the line function $\ell_{T,Q}(P)$ passing through T and Q and evaluated at P , as well as the sum $T + Q$, before updating the Miller function f . The algorithm can be further optimized since BLS12-381 has degree-6 twists. This implies that $\mathbb{G}_2 = E^t(\mathbb{F}_{p^2})[r] \cap \ker(\pi_p - [p])$, where $E^t/\mathbb{F}_{p^2}$ is the degree-6 twist of E . Consequently, the doubling and addition steps can be executed mainly over $\mathbb{F}_{p^2}$ instead of $\mathbb{F}_{p^{12}}$. Furthermore, since the embedding degree is even and the degree-6 twist is used, denominator elimination applies and, therefore, the vertical lines are excluded from the computation [Ver10]. The Miller loop returns the Miller function f , or its inverse $1/f$ if the seed z is negative (as is actually the case for the BLS12-381 curve), whereby the inversion can be efficiently computed via a conjugation. A high-level description of the Miller loop is given in Algorithm 1.

The most efficient technique for computing the final exponentiation on BLS12-381 is described in [HHT20]. The usual strategy is to split the process in two parts, namely an *easy part* and a *hard part*, which suggests the following decomposition:

$$\frac{p^{12}-1}{r} = \underbrace{(p^6-1)(p^2+1)}_{\text{easy part}} \underbrace{(p^4-p^2+1)/r}_{\text{hard part}}.$$

The easy part can be efficiently computed via one $\mathbb{F}_{p^{12}}$ multiplication, one inversion, and one conjugation. However, the hard part of the final exponentiation is significantly more

computation-intensive. The currently most widely-used formula for computing the hard part is due to Hayashida, Hayasaka, and Teruya [HHT20, Theorem 4] and obtains the exponent via

$$3 \cdot \frac{\Phi_{12}(p)}{r} = 3 \cdot \frac{p^4 - p^2 + 1}{r} = (z - 1)^2 \cdot (z + p) \cdot (z^2 + p^2 - 1) + 3 \quad (1)$$

$\mathbb{F}_{p^{12}}$ tower extension. The `blst` library adopts two different field towering options in different parts of the implementation and for different purposes. Both tower extensions construct the $\mathbb{F}_{p^{12}}$ extension from $\mathbb{F}_p$, using intermediate sub-fields of a certain degree, so that operations in all sub-fields are efficient.

The first tower extension is used for extension field operations in both the Miller loop and the final exponentiation; it can be summarized as $\mathbb{F}_p \rightarrow \mathbb{F}_{p^2} \rightarrow \mathbb{F}_{p^6} \rightarrow \mathbb{F}_{p^{12}}$, where each extension field is defined as follows:

$$\begin{aligned} \mathbb{F}_{p^2} &= \mathbb{F}_p[u]/\langle u^2 - \xi \rangle, \quad \text{where } \xi \in \mathbb{F}_p \\ \mathbb{F}_{p^6} &= \mathbb{F}_{p^2}[v]/\langle v^3 - \theta \rangle, \quad \text{where } \theta \in \mathbb{F}_{p^2} \\ \mathbb{F}_{p^{12}} &= \mathbb{F}_{p^6}[w]/\langle w^2 - \psi \rangle, \quad \text{where } \psi \in \mathbb{F}_{p^6} \end{aligned}$$

The usual setting for BLS12-381 is $\xi = -1 \in \mathbb{F}_p$, $\theta = u + 1 \in \mathbb{F}_{p^2}$, and $\psi = v \in \mathbb{F}_{p^6}$, which implies the identities $u^2 = -1$, $v^3 = u + 1$, and $w^2 = v$. Then, an element in $\mathbb{F}_{p^{12}}$ can be represented as $\alpha = a + a'w \in \mathbb{F}_{p^{12}}$, with $a, a' \in \mathbb{F}_{p^6}$. Hence, $a = a_0 + a_1v + a_2v^2$ and $a' = a_3 + a_4v + a_5v^2$, where $a_i \in \mathbb{F}_{p^2}$ with $i = 0, 1, 2, 3, 4, 5$. This implies that each a_i is of the form $a_i = a_{i,0} + a_{i,1}u$, where $a_{i,j} \in \mathbb{F}_p$, for every $i = 0, 1, 2, 3, 4, 5$ and $j = 0, 1$.

The second tower extension is $\mathbb{F}_p \rightarrow \mathbb{F}_{p^2} \rightarrow \mathbb{F}_{p^4} \rightarrow \mathbb{F}_{p^{12}}$, which is only used in the final exponentiation, in particular in the cyclotomic squaring. Each field in this towering is defined as follows:

$$\begin{aligned} \mathbb{F}_{p^2} &= \mathbb{F}_p[u]/\langle u^2 - \xi \rangle, \quad \text{where } \xi \in \mathbb{F}_p \\ \mathbb{F}_{p^4} &= \mathbb{F}_{p^2}[v]/\langle v^2 - \theta \rangle, \quad \text{where } \theta \in \mathbb{F}_{p^2} \\ \mathbb{F}_{p^{12}} &= \mathbb{F}_{p^4}[w]/\langle w^3 - \psi \rangle, \quad \text{where } \psi \in \mathbb{F}_{p^4} \end{aligned}$$

For the BLS12-381 curve, we set $\xi = -1 \in \mathbb{F}_p$, $\theta = u + 1 \in \mathbb{F}_{p^2}$, and $\psi = v \in \mathbb{F}_{p^4}$. The fact that $p^2 \equiv 1 \pmod{6}$ in BLS12-381 allows one to exploit the Squaring-Friendly Field (SFF) framework of Granger and Scott [GS10]. Therefore, an efficient squaring operation can be adopted in the cyclotomic sub-group $\mathbb{G}_{\Phi_6}(\mathbb{F}_{p^2}^*)$ of $\mathbb{F}_{p^{12}}^*$, which significantly speeds up the final exponentiation of the OAP on the BLS12-381 curve.

2.2 AVX extensions on Ice Lake CPUs

Ice Lake CPUs support different generations of AVX, including AVX2, AVX-512, as well as optional extensions of the latter, such as AVX-512IFMA. We focus primarily on AVX-512IFMA because of its massive computational power. While many of our optimizations for higher-level operations are relevant for both AVX-512IFMA and AVX2, the low-level functions need to be optimized individually due to the different limb sizes (48 vs 28 bits) and number of limbs (8 vs 14, see Section 1), which impacts vectorization strategies.

The AVX-512IFMA instruction set comprises the two instructions `vpmadd52luq` and `vpmadd52huq`, which are capable to multiply the low 52 bits of the eight 64-bit elements of a first 512-bit register `zmm1` by the low 52 bits of the eight 64-bit elements of a second 512-bit register `zmm2` and add either the lower halves or the higher halves of the resulting eight 104-bit products to the eight 64-bit elements of a 512-bit register `zmm3`. Limiting the operand lengths to 52 bits allows these multiplications to be integrated into existing

double-precision floating-point multipliers. Apart from the official Intel documents⁵, there are also third-party reports and analyses, which offer additional information about these instructions; a well-known example is the manual by Fog⁶. Abel and Reineke [AR19] give a more complete analysis, offering access to all data via an associated front-end⁷. On a 10th generation Intel Core CPU based on “Ice Lake” micro-architecture, such as the i3-1005G1 we used for benchmarking, the 512-bit IFMA instructions have a throughput of one cycle (when both operands are in `zmm` registers) and a latency of four cycles. Ice Lake based CPUs have eight execution ports, of which the ports 0, 1, and 5 can handle AVX instructions. The IFMA instructions `vpmadd52luq` and `vpmadd52huq` (with 512-bit vectors as operands) can only be dispatched through port 0. Thus, some other AVX-512 instruction (e.g., an addition or a shift) can be executed in parallel to IFMA, provided there are no dependencies preventing instruction-level parallelism.

Notation. When analyzing computational costs, i.e., the number of operations, we use M_i , S_i , A_i , and I_i to denote multiplication, squaring, addition/subtraction, and inversion in $\mathbb{F}_{p^i}$, respectively. We define a $(w \times x \times y \times z)$ -way vectorized implementation for an operation at the $\mathbb{F}_{p^6}$ (or $\mathbb{F}_{p^4}$) layer to perform w $\mathbb{F}_{p^6}$ (or $\mathbb{F}_{p^4}$) operations in parallel, with each of them performing x $\mathbb{F}_{p^2}$ operations in parallel, and each of these $\mathbb{F}_{p^2}$ operations executes y $\mathbb{F}_p$ operations in parallel, and each of the $\mathbb{F}_p$ operation uses z 64-bit elements of a vector. Similarly, for operations at the $\mathbb{F}_{p^2}$ and $\mathbb{F}_p$ layers, we define, respectively, an $(x \times y \times z)$ -way and a $(y \times z)$ -way vectorized implementation. Algorithmic descriptions in this paper use a single letter (e.g., t) to represent an operand having a similar length as a field-element, i.e., a single-length operand, while a double letter (e.g., tt) denotes an operand of roughly twice the length of a field-element, i.e., a double-length operand.

3 Implementation

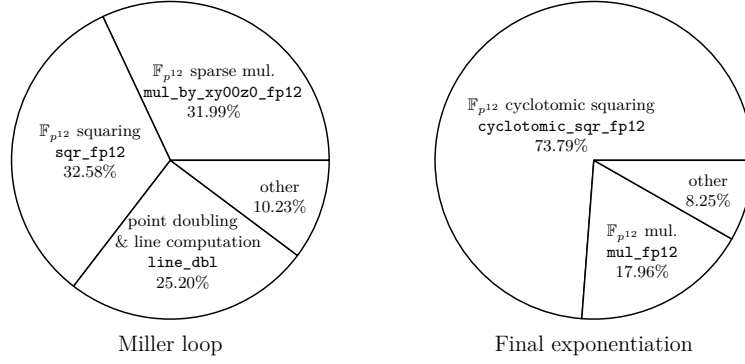
Target operations for vectorization. As indicated by Algorithm 1 and Equation 1, the operations in both the Miller loop and final exponentiation exhibit a strong data dependency. This means they *are not* SIMD-friendly, i.e., Algorithm 1 and Equation 1 can not be easily vectorized with the help of SIMD instructions. However, the elliptic curve and $\mathbb{F}_{p^{12}}$ sub-operations they use internally *are* SIMD-friendly: in this work, we therefore focus on said sub-operations. Figure 1 reports the proportion of execution time for the elliptic curve and $\mathbb{F}_{p^{12}}$ sub-routines used by `blst` to compute the Miller loop and final exponentiation. More specifically, we focus on six performance-critical sub-operations (which include the five sub-routines listed in Figure 1): these are 1) the $\mathbb{F}_{p^{12}}$ cyclotomic squaring (Section 3.1) and 2) the $\mathbb{F}_{p^{12}}$ multiplication (Section 3.2) carried out in the final exponentiation, 3) the $\mathbb{F}_{p^{12}}$ multiplication with a sparse element (Section 3.3) and 4) the $\mathbb{F}_{p^{12}}$ squaring (Section 3.4) used in the Miller loop, as well as 5) the `line_dbl` function to compute point doubling and the line $\ell_{T,T}$ (Section 3.5) plus 6) the `line_add` function to compute point addition and the line $\ell_{T,Q}$ (Section 3.6).

Vectorization strategy for $\mathbb{F}_p$ and $\mathbb{F}_{p^2}$ layers. The authors of [CFGR22, Section 3+4] analyzed different vectorization strategies for prime-field and quadratic extension-field operations. Although the prime p is different, their techniques can be applied to our case straightforwardly. Therefore, we developed our implementation of $\mathbb{F}_p$ and $\mathbb{F}_{p^2}$ operations based on the implementation of [CFGR22], with some necessary modifications due to the

⁵<https://www.intel.com/content/www/us/en/content-details/671104/10th-generation-intel-core-processor-instruction-throughput-and-latency-docs.html>

⁶https://www.agner.org/optimize/instruction_tables.pdf

⁷<https://uops.info/table.html>

Figure 1: Profiling of the OAP implementation in `blst` on an Intel Core i3-1005G1 CPU.

different prime p and our 48-bit-per-limb representation⁸ for elements of $\mathbb{F}_p$. Likewise, in this work, we solely consider (8×1) -way and (4×2) -way $\mathbb{F}_p$ implementations, the two most efficient options to vectorize $\mathbb{F}_p$ arithmetic. We conclude with an observation from [CFGR22], which can (and for us did) help develop more efficient AVX-512 software:

Observation 1. If no lanes are idle⁹, vectorization at the $\mathbb{F}_p$ layer has more impact on performance than vectorization at any higher layer. This is exemplified by the results in [CFGR22, Table 2], according to which a $(4 \times 2 \times 1)$ -way version of $\mathbb{F}_{p^2}$ multiplication is faster than a $(4 \times 1 \times 2)$ -way variant. This suggests that, for $\mathbb{F}_p$ operations, using an (8×1) -way vectorization should be preferred over alternatives.

3.1 $\mathbb{F}_{p^{12}}$ cyclotomic squaring

We start with the most performance-critical operation, namely the squaring in the cyclotomic sub-group $\mathbb{G}_{\Phi_6}(\mathbb{F}_{p^2})$ of $\mathbb{F}_{p^{12}}^*$, which is part of the final exponentiation routine. The cyclotomic squaring differs from the other three target $\mathbb{F}_{p^{12}}$ operations in that it adopts a different tower extension, in particular the $\mathbb{F}_p \rightarrow \mathbb{F}_{p^2} \rightarrow \mathbb{F}_{p^4} \rightarrow \mathbb{F}_{p^{12}}$ tower described in Section 2.1. The state-of-the-art technique for its computation is due to Granger and Scott [GS10], with a cost of $3S_4$. More concretely, let $\alpha = a + bw + cw^2 \in \mathbb{F}_{p^{12}}$ be an operand such that $a, b, c \in \mathbb{F}_{p^4}$. The square $\gamma = \alpha^2 = A + Bw + Cw^2 \in \mathbb{F}_{p^{12}}$ is computed via the relations $A = 3a^2 - 2\bar{a}$, $B = 3c^2v + 2\bar{b}$ and $C = 3b^2 - 2\bar{c}$, where $A, B, C \in \mathbb{F}_{p^4}$ and $\bar{a} = a^{p^2}$, $\bar{b} = b^{p^2}$, and $\bar{c} = c^{p^2}$ are the $\mathbb{F}_{p^2}$ -conjugates. We note that `blst` uses the Granger-Scott method for all squaring operations in $\mathbb{F}_{p^{12}}$ in the final exponentiation.

Compressed cyclotomic squaring. The *compressed cyclotomic squaring* was proposed in [AKL⁺11] as a way to further speed up the squaring operation in $\mathbb{G}_{\Phi_6}(\mathbb{F}_{p^2})$, in cases where multiple consecutive squarings are computed. The compressed version omits the constant term a in the representation of $\alpha \in \mathbb{F}_{p^{12}}$ given above, which then simplifies to $C(\alpha) = bw + cw^2$. Thus, the compressed method needs only the $\mathbb{F}_{p^4}$ elements B and C to

⁸A field-element is 381 bits long and consists of eight limbs if the maximum limb-length supported by AVX-512IFMA, i.e., 52 bits, is used. However, with 48-bit limbs, the number of limbs remains the same, namely eight, but each limb has more “headroom”, which simplifies some special techniques like Karatsuba multiplication. Namely, due to the 52×52 -bit multipliers of AVX-512IFMA, it is necessary to ensure that no limb exceeds 52 bits before being multiplied. However, in Karatsuba multiplication, some limb-wise multiplications must be executed after limb-wise additions, and these additions can increase the length of the limbs. Using a 52-bit-per-limb representation will, therefore, introduce some overhead to shorten the limbs to 52 bits (see [CFGR22, Section 3.1] for more details).

⁹A lane is termed “idle” if, during the execution of a vectorized function, it computes a result that is not used by a subsequent function.

```

1  /**
2  * Input:  a  = a1 | a0          in (2x2x2)-way
3  * Input:  bc = c1 | c0 | b1 | b0 in (4x2x1)-way
4  * Output: ra = A1 | A0          in (2x2x2)-way
5  * Output: rbc = C1 | C0 | B1 | B0 in (4x2x1)-way
6  */
7
8  /* Compute A in (1x2x2x2)-way */
9  sqr_fp4_1x2x2x2w(ta, a);          //          t1 |          t0
10 as_fp2_2x2x2w(ra, ta, a);          //          t1+a1 |          t0-a0
11 add_fp2_2x2x2w(ra, ra, ra);          // 2*(t1+a1) | 2*(t0-a0)
12 add_fp2_2x2x2w(ra, ra, ta);          // 3*t1+2*a1 | 3*t0-2*a0
13
14 /* Compute B and C simultaneously in (2x2x2x1w)-way */
15 sqr_fp4_2x2x2x1w(tbc, bc);          //          t5 |          t4 |          t3 |          t2
16 mul_by_up1_fp2_4x2x1w(tmp, tbc);    //  t5*(u+1) |  t4*(u+1) |  t3*(u+1) |  t2*(u+1)
17 // Transformation to make tbc =      //          t3 |          t2 |          t4 |          t5*(u+1)
18 assa_fp2_4x2x1w(rbc, tbc, bc);        //          t3+c1 |          t2-c0 |          t4-b1 |          t5*(u+1)+b0
19 add_fp2_4x2x1w(rbc, rbc, rbc);        // 2*(t3+c1) | 2*(t2-c0) | 2*(t4-b1) | 2*(t5*(u+1)+b0)
20 add_fp2_4x2x1w(rbc, rbc, tbc);        // 3*t3+2*c1 | 3*t2-2*c0 | 3*t4-2*b1 | 3*t5*(u+1)+2*b0

```

Figure 2: $\mathbb{F}_{p^{12}}$ cyclotomic squaring with a hybrid vectorization. The sub-routines for the vector transformation between `tbc` and `tmp`, which should be performed at line 17, are omitted for simplicity.

compute $C(\gamma) = Bw + Cw^2$, saving $1S_4$. The value A is retrieved via the decompression process at the cost of $9M_2 + 6S_2 + 22A_2 + I_2$, hence we get $\gamma = D(C(\gamma))$ (see [AKL⁺11] for details), whereby the $\mathbb{F}_{p^2}$ inversion adds a significant burden to the decompression function. Assuming $B = B_0 + B_1v \in \mathbb{F}_{p^4}$ with $B_0, B_1 \in \mathbb{F}_{p^2}$, the decompression process adopts a different formula, depending on whether $B_0 \neq 0$ or $B_0 = 0$. The compressed cyclotomic squaring approach is beneficial when raising an $\mathbb{F}_{p^{12}}$ element to the seed z in the hard part of the final exponentiation. This computation is carried out through the classic square-and-multiply method and requires $\lfloor \log_2(z) \rfloor$ compressed squarings, while the number of $\mathbb{F}_{p^{12}}$ multiplications depends on the Hamming weight of z . According to [AKL⁺11], only one decompression is needed, which can retrieve the missing $\mathbb{F}_{p^4}$ components simultaneously. Adopting the description in [AKL⁺11] to the BLS12-381 case, the computation of $g^{|z|}$, where $z = -(2^{63} + 2^{62} + 2^{60} + 2^{57} + 2^{48} + 2^{16})$, involves three steps:

1. Compute $C(g^{2^i})$ for $1 \leq i \leq 63$, and store the values for $i \in \{16, 48, 57, 60, 62, 63\}$.
2. Compute in parallel $D(C(g^{2^i}))$ for all $i \in \{16, 48, 57, 60, 62, 63\}$.
3. Compute the element $g^{|z|} = g^{2^{63}} \cdot g^{2^{62}} \cdot g^{2^{60}} \cdot g^{2^{57}} \cdot g^{2^{48}} \cdot g^{2^{16}}$.

Consequently, the majority of the operations for raising g to the exponent $|z|$ are the consecutive $\mathbb{F}_{p^{12}}$ compressed cyclotomic squarings.

We adapted `blst` by replacing only the Granger-Scott squarings used for computing $g^{|z|}$ with this compressed variant. Our experiments demonstrate a 15% improvement in the case of the scalar implementation of the final exponentiation with the compressed cyclotomic squaring and using the implementation of $\mathbb{F}_{p^2}$ inversion already contained in `blst`. However, note that for the specific implementation of the compressed cyclotomic squaring, we follow the implementation strategy of the RELIC library [AGM⁺] (i.e., the same implementation used by [Lon23]), which does not consider the case where $B_0 = 0$ in the decompression process. The reason behind this omission is that $B_0 = 0$ appears with negligible probability, and if an implementation includes this case while aiming to maintain constant-time execution, it will introduce significant overheads. Therefore, we consider this implementation of the compressed cyclotomic squaring as incomplete from this perspective. We implemented and used both versions of the cyclotomic squaring in `avxbls` and report the speed-up achieved by both implementations in Section 4. Note that, due to the dependency of internal operations, the $\mathbb{F}_{p^2}$ inversion is not vectorized.

Algorithm 2: $\mathbb{F}_{p^4}$ squaring with 2-way parallelization at $\mathbb{F}_{p^2}$ level, except only $\mathbb{F}_{p^2}$ multiplication in 1-way.

Input: $\mathbb{F}_{p^4}$ element $a = a_0 + a_1v$
Output: $\mathbb{F}_{p^4}$ element $r = r_0 + r_1v = (a_0^2 + a_1^2(u+1)) + 2a_0a_1v$

```

1  $tt_0 \leftarrow a_1^2$                                  $ss_0 \leftarrow a_0^2$                 /*  $(2x \times 2 \times z)$ -way */
2  $tt_0 \leftarrow tt_0 \cdot (u+1)$ 
3  $tt_1 \leftarrow a_0 \cdot a_1$                         /*  $(x \times 4 \times z)$ -way */
4  $tt_1 \leftarrow tt_1 + tt_0$                          $ss_0 \leftarrow ss_0 + tt_0$ 
5  $r_1 \leftarrow tt_1 \bmod p$                          $r_0 \leftarrow ss_0 \bmod p$ 
6 return  $r = r_0 + r_1v$ 

```

Vectorization strategy for $\mathbb{F}_{p^{12}}$ layer. The Granger-Scott squaring is not really well-suited for 2-way or 4-way vectorization because the number of underlying $\mathbb{F}_{p^4}$ squarings is three. However, since the operations for computing the coefficients A , B , and C are relatively independent, we developed a *hybrid* vectorization, shown in Figure 2, i.e., one coefficient is computed in 1-way at the $\mathbb{F}_{p^4}$ layer, while the other two are parallelized in 2-way. A key aspect in this hybrid strategy is the choice of the coefficient that will be computed independently of the other two. The reason why we decided to separate the sub-routines associated with A is that the computation of A requires only a , whereas the computation of B or C needs both b and c , hence it saves more vector transformations compared to the other options. On the other hand, the compressed cyclotomic squaring (computing only B and C) is vectorized as shown in lines 14-20 of Figure 2.

Vectorization strategy for $\mathbb{F}_{p^4}$ layer. Per Figure 2, the only $\mathbb{F}_{p^4}$ operation we need to vectorize is $\mathbb{F}_{p^4}$ squaring (line 9 and 15), i.e., $r_0 + r_1v = (a_0 + a_1v)^2 = (a_0^2 + a_1^2(u+1)) + 2a_0a_1v$. This operation costs $2S_2 + M_2$ (or $3S_2$ via Karatsuba’s method), but can not be well vectorized in 2-way or 4-way. We note that schoolbook-based $\mathbb{F}_{p^4}$ squaring is more efficient than the Karatsuba-based version for vectorized implementation since it needs fewer additions/subtractions. Moreover, we take advantage of the hybrid vectorization method as shown in Algorithm 2. Concretely, we use $(2x \times 2 \times z)$ -way parallelism (based on [CFGR22, Algorithm 7]) to compute two $\mathbb{F}_{p^2}$ squarings at line 1 and $(x \times 4 \times z)$ -way parallelism (based on [CFGR22, Algorithm 5]) for the $\mathbb{F}_{p^2}$ multiplication at line 3.

Lazy reduction. Apart from the vectorization strategy, lazy reduction is another issue that impacts the efficiency of $\mathbb{F}_{p^4}$ squaring (and of $\mathbb{F}_{p^6}$ operations described later). When $\mathbb{F}_{p^4}$ squaring is implemented straightforwardly (i.e., without lazy reduction), each of the underlying $\mathbb{F}_{p^2}$ operations always performs some modular reductions to ensure both the real and imaginary parts of the result are in $[0, p-1]$, making the $\mathbb{F}_{p^2}$ intermediates (in $\mathbb{F}_{p^4}$ squaring) to have *single length*. Alternatively, lazy reduction omits such reductions for certain $\mathbb{F}_{p^2}$ operations, especially multiplication and squaring. It defers the reduction until it becomes strictly necessary, such as the end of $\mathbb{F}_{p^4}$ squaring, which causes some $\mathbb{F}_{p^2}$ intermediates to adopt a *double-length* representation.

For scalar (i.e., non-vectorized) implementations, lazy reduction has been shown to be more efficient since it saves some costly modular reductions; for example, `blst` adopts lazy reduction for $\mathbb{F}_{p^4}$ squaring. On the other hand, for vectorized implementations, using lazy reduction would necessitate to perform vector transformations, such as blending and permutation, *frequently*, while they are not needed by scalar implementations at all. As double-length intermediates also require more execution time for vector transformations than single-length ones (due to the increased number of limbs), it is not clear whether lazy reduction can achieve a speed-up in vectorized settings. To answer this question, we

Algorithm 3: $\mathbb{F}_{p^6}$ multiplication with 2-way parallelization at $\mathbb{F}_{p^2}$ level.

Input: $\mathbb{F}_{p^6}$ elements $a = a_0 + a_1v + a_2v^2$ and $b = b_0 + b_1v + b_2v^2$

Output: $\mathbb{F}_{p^6}$ element $r = r_0 + r_1v + r_2v^2$

1 $t_0 \leftarrow b_1 + b_2$	$s_0 \leftarrow a_1 + a_2$
2 $t_1 \leftarrow b_0 + b_1$	$s_1 \leftarrow a_0 + a_1$
3 $t_2 \leftarrow b_0 + b_2$	$s_2 \leftarrow a_0 + a_2$
4 $tt_0 \leftarrow a_1 \cdot b_1$	$ss_0 \leftarrow a_0 \cdot b_0$
5 $tt_1 \leftarrow a_2 \cdot b_2$	$ss_1 \leftarrow s_0 \cdot t_0$
6 $tt_2 \leftarrow s_1 \cdot t_1$	$ss_2 \leftarrow s_2 \cdot t_2$
7 $tt_2 \leftarrow tt_2 - tt_0$	$ss_1 \leftarrow ss_1 - tt_0$
8 $ss_2 \leftarrow ss_2 - tt_1$	$ss_1 \leftarrow ss_1 - tt_1$
9 $tt_2 \leftarrow tt_2 - ss_0$	$ss_2 \leftarrow ss_2 - ss_0$
10 $tt_1 \leftarrow tt_1 \cdot (u + 1)$	$ss_1 \leftarrow ss_1 \cdot (u + 1)$
11 $r_1 \leftarrow tt_1 + tt_2$	$r_0 \leftarrow ss_0 + ss_1$
12 $r_2 \leftarrow ss_2 + tt_0$	
13 return $r = r_0 + r_1v + r_2v^2$	

developed both versions, and our experiments confirm that using lazy reduction is more efficient for $\mathbb{F}_{p^4}$ squaring, as well as for other $\mathbb{F}_{p^6}$ operations.

3.2 $\mathbb{F}_{p^{12}}$ multiplication

The multiplication in $\mathbb{F}_{p^{12}}$ is another crucial operation of the final exponentiation. Given two $\mathbb{F}_{p^{12}}$ elements $\alpha = a + a'w$ and $\beta = b + b'w$, the result can be computed as:

$$\gamma = r + r'w = \alpha \cdot \beta = \begin{cases} (ab + a'b'v) + (ab' + a'b)w & \text{(schoolbook)} \\ (ab + a'b'v) + ((a + a')(b + b') - ab - a'b')w & \text{(Karatsuba)} \end{cases}$$

Vectorization strategy for $\mathbb{F}_{p^{12}}$ layer. Since it saves an expensive $\mathbb{F}_{p^6}$ multiplication versus the schoolbook method, Karatsuba's method is faster for scalar implementation (and indeed used by `blst`). However, for vectorized implementation, the situation is less obvious, as the schoolbook method can compute ab , ab' , $a'b$, $a'b'$ in parallel and enable a 4-way parallelization at $\mathbb{F}_{p^6}$ level. This allows, at the lowest level, the $\mathbb{F}_p$ operation to be implemented in (8×1) -way, which, according to Observation 1, is very important from a performance perspective. To identify the most efficient option, we implemented both schoolbook and Karatsuba-based $\mathbb{F}_{p^{12}}$ multiplication, using the 2-way $\mathbb{F}_{p^6}$ multiplication described below. Our experimental results show that the Karatsuba-based approach is slightly faster; therefore, it is adopted for $\mathbb{F}_{p^{12}}$ multiplication in `avxbls`. The products ab and $a'b'$ are first computed in parallel (i.e., in 2-way at the $\mathbb{F}_{p^6}$ layer), followed by the computation of $(a + a')(b + b')$ in 1-way, meaning hybrid vectorization is again used.

Vectorization strategy for $\mathbb{F}_{p^6}$ layer. The central operation of multiplication in $\mathbb{F}_{p^{12}}$ is $\mathbb{F}_{p^6}$ multiplication. Given the $\mathbb{F}_{p^6}$ elements $a = a_0 + a_1v + a_2v^2$ and $b = b_0 + b_1 + b_2v^2$, we can compute the coefficients of the result $r = r_0 + r_1v + r_2v^2 = a \cdot b$ by using either the 3-way Karatsuba method [WP06] (on the left) or the schoolbook method (on the right):

$$\begin{aligned} r_0 &= ((a_1 + a_2)(b_1 + b_2) - a_1b_1 - a_2b_2)(u + 1) + a_0b_0 &= (a_1b_2 + a_2b_1)(u + 1) + a_0b_0 \\ r_1 &= (a_0 + a_1)(b_0 + b_1) - a_0b_0 - a_1b_1 + a_2b_2(u + 1) &= a_0b_1 + a_1b_0 + a_2b_2(u + 1) \\ r_2 &= (a_0 + a_2)(b_0 + b_2) - a_0b_0 - a_2b_2 + a_1b_1 &= a_0b_2 + a_2b_0 + a_1b_1 \end{aligned}$$

As shown above, the schoolbook method needs nine $\mathbb{F}_{p^2}$ multiplications. By contrast, the Karatsuba method not only reduces the number of $\mathbb{F}_{p^2}$ multiplication to six, but is also

Algorithm 4: Two $\mathbb{F}_{p^6}$ sparse multiplications (combined in one routine) with 2-way parallelization at $\mathbb{F}_{p^2}$ level.

Input: $\mathbb{F}_{p^6}$ elements $a = a_0 + a_1v + a_2v^2$, $b = b_0 + b_1v$, and $b' = b_4v$

Output: $\mathbb{F}_{p^6}$ elements $r = r_0 + r_1v + r_2v^2$ and $r' = r_3 + r_4v + r_5v^2$

1 $tt_0 \leftarrow a_2 \cdot b_4$	$ss_0 \leftarrow a_2 \cdot b_1$
2 $r_3 \leftarrow tt_0 \cdot (u + 1)$	$ss_0 \leftarrow ss_0 \cdot (u + 1)$
3 $r_5 \leftarrow a_1 \cdot b_4$	$r_4 \leftarrow a_0 \cdot b_4$
4 $tt_1 \leftarrow a_1 \cdot b_1$	$ss_1 \leftarrow a_0 \cdot b_0$
5 $t_0 \leftarrow b_0 + b_1$	$s_0 \leftarrow a_0 + a_1$
6 $tt_0 \leftarrow s_0 \cdot t_0$	$ss_2 \leftarrow a_2 \cdot b_0$
7 $tt_0 \leftarrow tt_0 - ss_1$	$r_2 \leftarrow ss_2 + tt_1$
8 $r_1 \leftarrow tt_0 - tt_1$	$r_0 \leftarrow ss_0 + ss_1$
9 return $r = r_0 + r_1v + r_2v^2, r' = r_3 + r_4v + r_5v^2$	

well-suited for 2-way vectorization, as illustrated in Algorithm 3. Consequently, we use Algorithm 3 to implement our $\mathbb{F}_{p^6}$ multiplication. More concretely, the $\mathbb{F}_{p^6}$ multiplication for computing ab and $a'b'$ is implemented in $(2 \times 2 \times 2 \times 1)$ -way, while the computation of $(a + a')(b + b')$ has to be carried out in $(1 \times 2 \times 2 \times 2)$ -way.

3.3 $\mathbb{F}_{p^{12}}$ multiplication with a sparse element

In the computation of the Miller loop, a sparse multiplication in $\mathbb{F}_{p^{12}}$ is carried out, where one operand α is an ordinary $\mathbb{F}_{p^{12}}$ element, while the other operand β , representing the line, is a sparse element such that $\beta = (b_0 + b_1v) + b_4vw$. An efficient way to compute it is to use two different underlying $\mathbb{F}_{p^6}$ multiplications, i.e., one $\mathbb{F}_{p^6}$ multiplication dedicated for $b_0 + b_1v$ and the other for b_4v , each of which is a sparse multiplication in $\mathbb{F}_{p^6}$. To be more concrete, with an ordinary $\mathbb{F}_{p^6}$ element $a_0 + a_1v + a_2v^2$, the first $\mathbb{F}_{p^6}$ multiplication $r_0 + r_1v + r_2v^2 = (a_0 + a_1v + a_2v^2) \cdot (b_0 + b_1v)$ is formally

$$\begin{aligned}
 r_0 &= ((a_1 + a_2)b_1 - a_1b_1)(u + 1) + a_0b_0 &= a_2b_1(u + 1) + a_0b_0 \\
 r_1 &= (a_0 + a_1)(b_0 + b_1) - a_0b_0 - a_1b_1 &= a_0b_1 + a_1b_0 \\
 r_2 &= (a_0 + a_2)b_0 - a_0b_0 + a_1b_1 &= a_2b_0 + a_1b_1
 \end{aligned}$$

and the second one $r_3 + r_4v + r_5v^2 = (a_0 + a_1v + a_2v^2) \cdot b_4v$ is computed via the relations $r_3 = a_2b_4(u + 1)$, $r_4 = a_0b_4$, and $r_5 = a_1b_4$.

Vectorization strategy for $\mathbb{F}_{p^{12}}$ layer. Similarly, there are schoolbook and Karatsuba options; **blst** uses the latter to save an $\mathbb{F}_{p^6}$ sparse multiplication that consists of three $\mathbb{F}_{p^2}$ multiplications. Since Karatsuba reduces only three $\mathbb{F}_{p^2}$ multiplications, while the schoolbook method provides a better parallelization at the $\mathbb{F}_{p^6}$ level, we take advantage of the schoolbook multiplication to implement the

Vectorization strategy for $\mathbb{F}_{p^6}$ layer. As per above, the number of underlying $\mathbb{F}_{p^2}$ multiplications executed in the two $\mathbb{F}_{p^6}$ sparse multiplications is, respectively, five (due to Karatsuba's method) and three, which means, contrary to a normal $\mathbb{F}_{p^6}$ multiplication (i.e., Algorithm 3), neither of them is ideal for a 2-way vectorization. However, when we combine these two $\mathbb{F}_{p^6}$ multiplications in one routine, there are eight $\mathbb{F}_{p^2}$ multiplications in total, which can be completely vectorized in 2-way, see Algorithm 4. Furthermore, the eight $\mathbb{F}_{p^2}$ multiplications are ideal for even 4-way; the associated vectorization is shown in Algorithm 5. Note that the $\mathbb{F}_{p^2}$ operation $a_i \cdot (u + 1)$ at the $\mathbb{F}_p$ level is essentially

$$a_i \cdot (u + 1) = (a_{i,0} + a_{i,1}u) \cdot (u + 1) = (a_{i,0} - a_{i,1}) + (a_{i,0} + a_{i,1})u,$$

Algorithm 5: Two $\mathbb{F}_{p^6}$ sparse multiplications (combined in one routine) with 4-way parallelization at $\mathbb{F}_{p^2}$ level.

Input: $\mathbb{F}_{p^6}$ elements $a = a_0 + a_1v + a_2v^2$, $b = b_0 + b_1v$, and $b' = b_4v$

Output: $\mathbb{F}_{p^6}$ elements $r = r_0 + r_1v + r_2v^2$ and $r' = r_3 + r_4v + r_5v^2$

```

1  $tt_0 \leftarrow a_2 \cdot b_1$             $ss_0 \leftarrow a_2 \cdot b_4$             $r_5 \leftarrow a_1 \cdot b_4$             $r_4 \leftarrow a_0 \cdot b_4$ 
2  $t_0 \leftarrow a_0 + a_1$           $s_0 \leftarrow b_0 + b_1$ 
3  $tt_1 \leftarrow a_0 \cdot b_0$       $ss_1 \leftarrow a_1 \cdot b_1$             $tt_2 \leftarrow a_2 \cdot b_0$             $ss_2 \leftarrow t_0 \cdot s_0$ 
4  $tt_0 \leftarrow tt_0 \cdot (u+1)$   $r_3 \leftarrow ss_0 \cdot (u+1)$         $r_2 \leftarrow tt_2 + ss_1$             $ss_2 \leftarrow ss_2 - tt_1$ 
5  $r_0 \leftarrow tt_0 + tt_1$         $r_1 \leftarrow ss_2 - ss_1$ 
6 return  $r = r_0 + r_1v + r_2v^2, r' = r_3 + r_4v + r_5v^2$ 

```

i.e., a composition of one $\mathbb{F}_p$ addition and one $\mathbb{F}_p$ subtraction, which can, therefore, be carried out simultaneously with $\mathbb{F}_{p^2}$ additions/subtractions.

This combined routine is beneficial for Karatsuba-based $\mathbb{F}_{p^{12}}$ sparse multiplication to compute ab and $a'b'$ at the beginning (in fact, this is ideal for our schoolbook-based implementation since it can be used to compute ab , ab' , $a'b$, and $a'b'$ in parallel).

$\mathbb{F}_{p^6}$ sparse multiplication: 2-way vectorization vs. 4-way vectorization. We developed two different $\mathbb{F}_{p^{12}}$ sparse multiplications based on, respectively, the 2-way vectorized $\mathbb{F}_{p^6}$ sparse multiplication (i.e., Algorithm 4) and the 4-way variant (i.e., Algorithm 5). It is somewhat counter-intuitive that, based on our experiments, the 2-way variant is more efficient than the 4-way one, but this can be explained as follows. Algorithm 4 vectorizes the $\mathbb{F}_{p^6}$ sparse multiplication in $(2 \times 2 \times 2 \times 1)$ -way, but in Algorithm 5 the vectorization is $(2 \times 4 \times 1 \times 1)$ -way. Both vectorizations use the (8×1) -way $\mathbb{F}_p$ implementation, the most efficient option. However, the 4-way variant requires more vector transformations and forces some lanes to be idle (concretely line 2 and 5 in Algorithm 5), making it less efficient than the 2-way variant. This example leads to another observation:

Observation 2. The importance of vectorization at a higher layer is to enable a more (or even the most) efficient $\mathbb{F}_p$ implementation at the bottom layer.

3.4 $\mathbb{F}_{p^{12}}$ squaring

The last arithmetic operation in $\mathbb{F}_{p^{12}}$ that needs to be vectorized is $\mathbb{F}_{p^{12}}$ squaring. Given an $\mathbb{F}_{p^{12}}$ element $\alpha = a + a'w$, the result can be computed as:

$$\gamma = r + r'w = \alpha^2 = \begin{cases} (a^2 + a'^2v) + 2aa'w & \text{(schoolbook)} \\ ((a + a')(a + a'v) - aa' - aa'v) + 2aa'w & \text{(Karatsuba)} \end{cases}$$

Vectorization strategy for $\mathbb{F}_{p^{12}}$ layer. As per above equation, the schoolbook method costs $2S_6 + M_6$, while the Karatsuba method has a cost of only $2M_6$. Apparently, the Karatsuba-based implementation is more efficient and, in addition, more vectorization-friendly. Thus, we use the Karatsuba method; our vectorized $\mathbb{F}_{p^{12}}$ squaring is shown in Algorithm 6. Notably, the two $\mathbb{F}_{p^6}$ multiplications at line 3 are implemented according to Algorithm 3 (internally vectorized in 2-way at the $\mathbb{F}_{p^2}$ level), plus $\mathbb{F}_{p^2}$ reductions for the results. This means that, overall, the $\mathbb{F}_{p^{12}}$ squaring is 4-way vectorized at the $\mathbb{F}_{p^2}$ level and, more importantly, the (8×1) -way $\mathbb{F}_p$ implementation is used at the bottom.

3.5 Doubling step

The doubling step is composed of four types of computation: point doubling, line computation, line evaluation, and update of the Miller function. The first three operations

Algorithm 6: $\mathbb{F}_{p^{12}}$ squaring with 4-way parallelization at $\mathbb{F}_{p^2}$ level.

Input: $\mathbb{F}_{p^{12}}$ element $\alpha = (a_0 + a_1v + a_2v^2) + (a_3 + a_4v + a_5v^2)w$ **Output:** $\mathbb{F}_{p^{12}}$ element $\gamma = (r_0 + r_1v + r_2v^2) + (r_3 + r_4v + r_5v^2)w$

```

1  $t_0 \leftarrow a_5 \cdot (u + 1)$        $s_0 \leftarrow a_2 + a_5$        $t_1 \leftarrow a_1 + a_4$        $s_1 \leftarrow a_0 + a_3$ 
2  $t_2 \leftarrow a_2 + a_4$        $s_2 \leftarrow a_1 + a_3$        $t_3 \leftarrow a_0 + t_0$ 
3  $(t_0, t_1, t_2) \leftarrow (a_0, a_1, a_2) \times_6 (a_3, a_4, a_5)$        $(s_0, s_1, s_2) \leftarrow (s_1, t_1, s_0) \times_6 (t_3, s_2, t_2)$ 
4  $r_5 \leftarrow t_2 + t_2$        $r_4 \leftarrow t_1 + t_1$        $r_3 \leftarrow t_0 + t_0$ 
5  $t_2 \leftarrow t_2 \cdot (u + 1)$        $s_2 \leftarrow s_2 - t_2$        $s_1 \leftarrow s_1 - t_1$        $s_0 \leftarrow s_0 - t_0$ 
6  $r_2 \leftarrow s_2 - t_1$        $r_1 \leftarrow s_1 - t_0$        $r_0 \leftarrow s_0 - t_2$ 
7 return  $\gamma = (r_0 + r_1v + r_2v^2) + (r_3 + r_4v + r_5v^2)w$ 
```

Algorithm 7: line_db1 with 2-way parallelization at $\mathbb{F}_{p^2}$ level. It computes the point $R = [2]T$ and the coefficients of the line $\ell_{T,T}$.

Input: Projective point $T = (X_T, Y_T, Z_T)$ **Output:** Projective point $R = (X_R, Y_R, Z_R)$ and line $\ell_{T,T} = (\ell_0, \ell_1, \ell_2)$

```

1  $B \leftarrow Y_T^2$       /*  $B = Y_T^2$  */       $A \leftarrow X_T^2$       /*  $A = X_T^2$  */
2  $t_0 \leftarrow X_T + B$       /*  $X_T + B$  */       $s_0 \leftarrow A + A$       /*  $2A$  */
3  $t_0 \leftarrow t_0^2$       /*  $(X_T + B)^2$  */       $C \leftarrow B^2$       /*  $C = B^2$  */
4  $t_0 \leftarrow t_0 - A$       /*  $(X_T + B)^2 - A$  */       $E \leftarrow s_0 + A$       /*  $E = 3A$  */
5  $t_0 \leftarrow t_0 - C$       /*  $(X_T + B)^2 - A - C$  */       $s_0 \leftarrow E + X_T$       /*  $3A + X_T$  */
6  $ZZ \leftarrow Z_T^2$       /*  $ZZ = Z_T^2$  */       $F \leftarrow E^2$       /*  $F = E^2$  */
7  $D \leftarrow t_0 + t_0$       /*  $D = 2((X_T + B)^2 - A - C)$  */       $s_1 \leftarrow Y_T + Z_T$       /*  $Y_T + Z_T$  */
8  $s_0 \leftarrow s_0^2$       /*  $(3A + X_T)^2$  */       $s_1 \leftarrow s_1^2$       /*  $(Y_T + Z_T)^2$  */
9  $t_0 \leftarrow F - D$       /*  $F - D$  */       $s_1 \leftarrow s_1 - B$       /*  $(Y_T + Z_T)^2 - B$  */
10  $X_R \leftarrow t_0 - D$       /*  $X_R = F - 2D$  */       $s_0 \leftarrow s_0 - A$       /*  $(3A + X_T)^2 - X_T^2$  */
11  $Z_R \leftarrow s_1 - ZZ$       /*  $Z_R = 2Y_T Z_T$  */       $s_2 \leftarrow C + C$       /*  $2C$  */
12  $s_2 \leftarrow s_2 + s_2$       /*  $4C$  */       $s_1 \leftarrow B + B$       /*  $2Y_T^2$  */
13  $s_2 \leftarrow s_2 + s_2$       /*  $8C$  */       $s_1 \leftarrow s_1 + s_1$       /*  $4Y_T^2$  */
14  $t_0 \leftarrow D - X_R$       /*  $D - X_R$  */       $s_0 \leftarrow s_0 - F$       /*  $(3A + X_T)^2 - X_T^2 - 9A^2$  */
15  $t_0 \leftarrow t_0 \cdot E$       /*  $E(D - X_R)$  */       $\ell_1 \leftarrow ZZ \cdot E$       /*  $\ell_1 = 3X_T^2 Z_T^2$  */
16  $Y_R \leftarrow t_0 - s_2$       /*  $Y_R = E(D - X_R) - 8C$  */       $\ell_0 \leftarrow s_0 - s_1$       /*  $\ell_0 = 6X_T^3 - 4Y_T^2$  */
17  $\ell_2 \leftarrow Z_R \cdot ZZ$       /*  $\ell_2 = Z_R Z_T^2$  */
18 return  $R = (X_R, Y_R, Z_R), \ell_{T,T} = (\ell_0, \ell_1, \ell_2)$ 
```

Algorithm 8: line_db1 with 4-way parallelization at $\mathbb{F}_{p^2}$ level. It computes the point $[2]T$ and the coefficients of the line $\ell_{T,T}$.

Input: Projective point $T = (X_T, Y_T, Z_T)$ **Output:** Projective point $R = (X_R, Y_R, Z_R)$ and line $\ell_{T,T} = (\ell_0, \ell_1, \ell_2)$

```

1  $t_0 \leftarrow Y_T + Z_T$ 
2  $B \leftarrow Y_T^2$        $A \leftarrow X_T^2$        $ZZ \leftarrow Z_T^2$        $s_0 \leftarrow t_0^2$ 
3  $t_1 \leftarrow X_T + B$        $s_1 \leftarrow A + A$        $t_2 \leftarrow B + B$        $s_2 \leftarrow A + X_T$ 
4  $t_0 \leftarrow s_1 + s_2$        $E \leftarrow s_1 + A$        $t_2 \leftarrow t_2 + t_2$ 
5  $t_1 \leftarrow t_1^2$        $C \leftarrow B^2$        $t_0 \leftarrow t_0^2$        $F \leftarrow E^2$ 
6  $t_1 \leftarrow t_1 + t_1$        $s_2 \leftarrow C + C$        $t_0 \leftarrow t_0 - A$        $s_0 \leftarrow s_0 - B$ 
7  $t_1 \leftarrow t_1 - s_2$        $s_2 \leftarrow s_2 + s_2$        $t_0 \leftarrow t_0 - F$        $Z_R \leftarrow s_0 - ZZ$ 
8  $D \leftarrow t_1 - s_1$        $s_2 \leftarrow s_2 + s_2$        $\ell_0 \leftarrow t_0 - t_2$ 
9  $t_0 \leftarrow F - D$        $s_0 \leftarrow D - F$        $t_1 \leftarrow D + D$ 
10  $X_R \leftarrow t_0 - D$        $s_0 \leftarrow s_0 + t_1$ 
11  $s_0 \leftarrow s_0 \cdot E$        $\ell_1 \leftarrow ZZ \cdot E$        $\ell_2 \leftarrow Z_R \cdot ZZ$ 
12  $Y_R \leftarrow s_0 - s_2$ 
13 return  $R = (X_R, Y_R, Z_R), \ell_{T,T} = (\ell_0, \ell_1, \ell_2)$ 
```

are executed over $\mathbb{F}_{p^2}$ and the last over $\mathbb{F}_{p^{12}}$. Considering the OAP on BLS12-381 and following the notation of Algorithm 1, let $T = (X_T, Y_T, Z_T) \in E^t(\mathbb{F}_{p^2})[r]$ be a point in projective coordinates on the twist $E^t/\mathbb{F}_{p^2}$ of BLS12-381. Then $R = [2]T$ is defined as:

$$R = (X_R, Y_R, Z_R) = (9X_T^4 - 8X_TY_T^2, 3X_T^2(12X_TY_T^2 - 9X_T^4) - 8Y_T^4, 2Y_TZ_T^3)$$

For the point doubling operation, **blst** implements the formula **dbl-2009-1** specified in the Explicit-Formulas Database (EFD)¹⁰. The line computation refers to the construction of the tangent line $\ell_{T,T}$ that passes through the point T , described by the coefficients ℓ_0, ℓ_1, ℓ_2 :

$$\ell_{T,T} = (\ell_0, \ell_1, \ell_2) = (6X_T^3 - 4Y_T^2, 3X_T^2Z_T^2, 2Y_TZ_T^3)$$

The computation of $R = [2]T$ and $\ell_{T,T}$ has a cost of $3M_2 + 8S_2 + 21A_2$. In Algorithm 7 (resp. Algorithm 8) we present our 2-way (resp. 4-way) implementation for computing $R = [2]T$ and the line $\ell_{T,T}$ (i.e., **line_dbl**). According to our experiments, compared to the 2-way variant, the 4-way implementation offers reduced latency and, therefore, it is integrated in our **avxbls**.

The evaluation of the line $\ell_{T,T}$ at the affine point $P = (x_P, y_P) \in E(\mathbb{F}_p)$ corresponds to the computation of the value:

$$\ell_{T,T}(P) = \ell_0 - 2x_P\ell_1w^2 + 2y_P\ell_2w^3 = (\ell_0 - 2x_P\ell_1v) + (2y_P\ell_2v)w,$$

which is a sparse element in $\mathbb{F}_{p^{12}}$ with respect to the extension field tower $\mathbb{F}_p \rightarrow \mathbb{F}_{p^2} \rightarrow \mathbb{F}_{p^6} \rightarrow \mathbb{F}_{p^{12}}$ in Section 2.1. When the values $-2x_P$ and $2y_P$ are precomputed and given as input in the OAP function, the computation of $\ell_{T,T}(P)$ requires $4M_1$ operations, which can be straightforwardly 4-way vectorized at the $\mathbb{F}_p$ level. The doubling step terminates by updating the Miller function as $f = f^2 \cdot \ell_{T,T}(P)$, which takes one squaring in $\mathbb{F}_{p^{12}}$ and one $\mathbb{F}_{p^{12}}$ multiplication with a sparse element. We refer to Section 3.4 and Section 3.3 for the description of our vectorization strategy for both operations.

Remark 1. For **line_dbl** (i.e., point doubling and line computation), faster alternatives might be available; for example, the algorithms presented in [AKL⁺11] and [CLN10] are more efficient compared to the one used in **blst** when considering x64 scalar implementation. **avxbls** adopts a 4-way implementation of the **blst** doubling step, which implies an (8×1) -way $\mathbb{F}_p$ implementation at the bottom. In the vectorized implementation, the formulas proposed in [AKL⁺11, CLN10] do not offer an efficiency advantage compared to the formula used in **blst** and, thus, we did not consider the vectorization of alternative formulas for point doubling and line computation.

3.6 Addition step

The addition step also consists of four operations: point addition, line computation, line evaluation, and update of the Miller function. The former three are executed in $\mathbb{F}_{p^2}$ and the latter over $\mathbb{F}_{p^{12}}$. Following the notation used in Algorithm 1, let $T = (X_T, Y_T, Z_T) \in E^t(\mathbb{F}_{p^2})[r]$ and $Q = (X_Q, Y_Q, Z_Q) \in E^t(\mathbb{F}_{p^2})[r]$ be points of order r on the twist $E^t/\mathbb{F}_{p^2}$ of BLS12-381. Then, $R = T + Q = (X_R, Y_R, Z_R)$ is defined as:

$$\begin{aligned} X_R &= 4(Y_QZ_T^3 - Y_T)^2 - 4(X_QZ_T^2 + X_T)(X_QZ_T^2 - X_T)^2 \\ Y_R &= 2(Y_QZ_T^3 - Y_T)(4X_T(X_QZ_T^2 - X_T)^2 - X_R) - 8Y_T(X_QZ_T^2 - X_T)^3 \\ Z_R &= 2Z_T(X_QZ_T^2 - X_T) \end{aligned}$$

For the point addition operation, **blst** implements the formula **madd-2007-b1** from the EFD¹¹. The line computation corresponds to the construction of the line $\ell_{T,Q}$ that passes

¹⁰<https://www.hyperelliptic.org/EFD/g1p/auto-shortw-jacobian-0.html#doubling-dbl-2009-1>.

¹¹<https://www.hyperelliptic.org/EFD/g1p/auto-shortw-jacobian-0.html#addition-madd-2007-b1>.

Algorithm 9: `line_add` with 2-way parallelization at $\mathbb{F}_{p^2}$ level. It computes the point $T + Q$ and the coefficients of the line $\ell_{T,Q}$.

Input: Projective point $T = (X_T, Y_T, Z_T)$ and affine point $Q = (X_Q, Y_Q)$

Output: Projective point $R = (X_R, Y_R, Z_R)$ and line $\ell_{T,Q} = (\ell_0, \ell_1, \ell_2)$

```

1  ZZ ← Z_T · Z_T          /* ZZ = Z_T^2 */  s0 ← Y_Q · Z_T          /* Y_Q Z_T */
2  U ← X_Q · ZZ            /* U = X_Q Z_T^2 */  S ← s0 · ZZ            /* S = Y_Q Z_T^3 */
3  H ← U - X_T            /* H = U - X_T */    s0 ← S - Y_T          /* S - Y_T */
4  t0 ← Z_T + H           /* Z_T + H */       r ← s0 + s0          /* r = 2(S - Y_T) */
5  t0 ← t0^2              /* (Z_T + H)^2 */    HH ← H^2            /* HH = H^2 */
6  t0 ← t0 - ZZ           /* (Z_T + H)^2 - Z_T^2 */ s0 ← HH + HH        /* 2H^2 */
7  Z_R ← t0 - HH          /* Z_R = 2Z_T H */   I ← s0 + s0          /* I = 4H^2 */
8  J ← H · I              /* J = HI */         s0 ← r · X_Q          /* r X_Q */
9  t0 ← r · r             /* r^2 */            s1 ← Y_Q · Z_R        /* Y_Q Z_R */
10 V ← X_T · I            /* V = X_T I */      s2 ← Y_T · J          /* Y_T J */
11 t0 ← t0 - J            /* r^2 - J */        s3 ← V + V            /* 2V */
12 X_R ← t0 - s3          /* X_R = r^2 - J - 2V */ s2 ← s2 + s2          /* 2Y_T J */
13 t0 ← V - X_R           /* V - X_R */        s0 ← s0 - s1          /* r X_Q - Y_Q Z_R */
14 t0 ← t0 · r            /* r(V - X_R) */
15 Y_R ← t0 - s2          /* Y_R = r(V - X_R) - 2Y_T J */ l0 ← s0 + s0          /* l0 = 2(r X_Q - Y_Q Z_R) */
16 l1 ← r                 /* l1 = r */          l2 ← Z_R              /* l2 = Z_R */
17 return R = (X_R, Y_R, Z_R), l_{T,Q} = (l0, l1, l2)

```

through T and Q and it is described by the coefficients ℓ_0, ℓ_1, ℓ_2 :

$$\ell_{T,Q} = (\ell_0, \ell_1, \ell_2) = (4X_Q(Y_Q Z_T^3 - Y_T) - 2Y_Q Z_R, 2(Y_Q Z_T^3 - Y_T), Z_R)$$

In summary, the process for computing the point $R = T + Q$ and the line $\ell_{T,Q}$ requires $9M_2 + 4S_2 + 16A_2$. Algorithm 9 illustrates our 2-way implementation for computing the point $R = T + Q$ and the line $\ell_{T,Q}$ (i.e., `line_add`). We note that, unlike to the doubling case, we were not able to derive a 4-way implementation for the point addition and line computation in the addition step that offers any efficiency advantages compared to the 2-way implementation.

The addition step requires the evaluation of the line $\ell_{T,Q}$ at the affine point $P = (x_P, y_P) \in E(\mathbb{F}_p)$. This corresponds to the value:

$$\ell_{T,Q}(P) = \ell_0 - 2x_P \ell_1 w^2 + 2y_P \ell_2 w^3 = (\ell_0 - 2x_P \ell_1 v) + (2y_P \ell_2 v)w,$$

which is a sparse element in $\mathbb{F}_{p^{12}}$ with respect to the extension field tower $\mathbb{F}_p \rightarrow \mathbb{F}_{p^2} \rightarrow \mathbb{F}_{p^6} \rightarrow \mathbb{F}_{p^{12}}$ in Section 2.1. When the values $-2x_P$ and $2y_P$ are precomputed and given as input in the OAP function, the computation of $\ell_{T,Q}(P)$ requires $4M_1$ operations, which can be straightforwardly 4-way vectorized at the $\mathbb{F}_p$ level. The addition step concludes by updating the Miller function as $f = f \cdot \ell_{T,Q}(P)$, which takes one $\mathbb{F}_{p^{12}}$ multiplication with a sparse element. We refer to Section 3.3 for the implementation details.

3.7 Integration

After vectorizing all target operations, whose optimization (i.e., vectorization) strategies are summarized in Table 1, the last step is to replace the corresponding x64 components by our AVX-512 functions, for which different types of transformation are needed. The first type is *data transformation*, i.e., the conversion of operands between radix-2⁶⁴ and radix-2⁴⁸ representation. To reduce the related overheads, we developed (8×1) -way and (4×2) -way radix conversions, which are not included in [CFGR22]. The second type is *vector transformation*, i.e., the conversion of vectors into the correct form needed by the

Table 1: Summary of optimization and vectorization strategies for $\mathbb{F}_{p^{12}}$, $\mathbb{F}_{p^6}$ and $\mathbb{F}_{p^4}$, and line operations in `avxb1s`.

Operation	Optimization	Vectorization
$\mathbb{F}_{p^{12}}$ cyclotomic squaring	Granger-Scott	hybrid parallelization at $\mathbb{F}_{p^2}$ level (per Figure 2), one/two parallel $\mathbb{F}_{p^4}$ squarings (i.e., Algorithm 2)
$\mathbb{F}_{p^{12}}$ multiplication	Karatsuba	hybrid parallelization at $\mathbb{F}_{p^2}$ level, one/two parallel $\mathbb{F}_{p^6}$ multiplications (i.e., Algorithm 3)
$\mathbb{F}_{p^{12}}$ sparse multiplication	schoolbook	4-way parallelization at $\mathbb{F}_{p^2}$ level, two parallel $\mathbb{F}_{p^6}$ sparse multiplications (i.e., Algorithm 4)
$\mathbb{F}_{p^{12}}$ squaring	Karatsuba	4-way parallelization at $\mathbb{F}_{p^2}$ level (per Algorithm 6), two parallel $\mathbb{F}_{p^6}$ multiplications (i.e., Algorithm 3)
$\mathbb{F}_{p^4}$ squaring	schoolbook, lazy reduction	hybrid parallelization at $\mathbb{F}_{p^2}$ level (per Algorithm 2)
$\mathbb{F}_{p^6}$ multiplication	Karatsuba, lazy reduction	2-way parallelization at $\mathbb{F}_{p^2}$ level (per Algorithm 3)
$\mathbb{F}_{p^6}$ sparse multiplication	Karatsuba, lazy reduction	2-way parallelization at $\mathbb{F}_{p^2}$ level (per Algorithm 4), combined routine
<code>line_dbl</code>	<code>dbl-2009-1</code>	4-way parallelization at $\mathbb{F}_{p^2}$ level (per Algorithm 8)
<code>line_add</code>	<code>madd-2007-b1</code>	2-way parallelization at $\mathbb{F}_{p^2}$ level (per Algorithm 9)

subsequent vector sub-routine. For example, in the Miller loop (Algorithm 1), all used operations are actually vectorized, and for each pair of different operations, we developed a dedicated transformation to efficiently convert the output vectors from the previous operation to the required form of the input vectors for the next operation. On the other hand, in the final exponentiation, a specific $\mathbb{F}_{p^{12}}$ exponentiation $g^{|z|}$ is carried out, where $g \in \mathbb{F}_{p^{12}}$ and $z \in \mathbb{Z}$ is the seed introduced in Section 2.1. According to Section 3.1, the majority of the underlying operations are the consecutive $\mathbb{F}_{p^{12}}$ (compressed) cyclotomic squarings. As indicated in Figure 2, the $\mathbb{F}_{p^2}$ coefficients in the output (`ra` and `rbc`) are arranged in the same form as those in the input (`a` and `bc`). This alignment enables the $\mathbb{F}_{p^{12}}$ (compressed) cyclotomic squaring operation to directly process the output of the previous one without incurring any overhead for vector transformation.

4 Evaluation

Benchmarking environment. As mentioned before, our benchmarking experiments were carried out on an Intel “Ice Lake” Core i3-1005G1 CPU, clocked at 1.2 GHz. The source codes of `blst` (x64 assembly), Longa’s implementation [Lon23] (also x64 assembly), and `avxb1s` (AVX-512) were compiled using GCC 13.3 with optimization level `-O3`. TurboBoost was disabled when measuring the execution time.

Comparison: `avxb1s`, `blst`, and Longa’s implementation. The benchmarking results of the low-level and the high-level arithmetic operations of the three software libraries are summarized in Table 2 and Table 3, respectively, whereby we use the non-vectorized (i.e., scalar) x64 assembly implementation of `blst` as the baseline for comparison. Since lazy reduction is widely used, Table 2 specifies whether an operation includes a reduction step or not¹². Note that `avxb1s` contains multiple vectorized implementations for a single low-level operation, which are needed by various vectorized high-level operations.

¹²A “without reduction” indicates that the operation does not include a reduction step, i.e., it adopts lazy reduction.

Table 2: Benchmarks of the low-level arithmetic operations in **blst** (x64 assembly implementation), Longa’s implementation [Lon23] (x64 assembly implementation), and **avxbls** (AVX-512 implementation) on an Intel Core i3-1005G1 CPU.

Operation	Library	Vectorization	#Instances	Cycles	Cyc./inst.	Speed-up
$\mathbb{F}_{p^2}$ multiplication (with reduction)	blst	1-way	1	409	409	1.00×
	[Lon23]	1-way	1	363	363	1.13×
	avxbls	$(4 \times 2 \times 1)$ -way	4	543	136	3.01×
	avxbls	$(2 \times 2 \times 2)$ -way	2	381	191	2.14×
$\mathbb{F}_{p^2}$ multiplication (without reduction)	blst	1-way	1	239	239	1.00×
	avxbls	$(8 \times 1 \times 1)$ -way	8	611	76	3.14×
	avxbls	$(4 \times 2 \times 1)$ -way	4	314	79	3.03×
	avxbls	$(2 \times 4 \times 1)$ -way	2	210	105	2.28×
	avxbls	$(2 \times 2 \times 2)$ -way	2	214	107	2.23×
	avxbls	$(1 \times 4 \times 2)$ -way	1	152	152	1.57×
$\mathbb{F}_{p^2}$ squaring (with reduction)	blst	1-way	1	302	302	1.00×
	[Lon23]	1-way	1	257	257	1.18×
	avxbls	$(4 \times 2 \times 1)$ -way	4	474	119	2.54×
	avxbls	$(2 \times 2 \times 2)$ -way	2	328	164	1.84×
$\mathbb{F}_{p^2}$ squaring (without reduction)	blst	1-way	1	162	162	1.00×
	avxbls	$(4 \times 2 \times 1)$ -way	4	238	60	2.70×
	avxbls	$(2 \times 2 \times 2)$ -way	2	140	70	2.31×
$\mathbb{F}_{p^4}$ squaring (with reduction)	blst	1-way	1	957	957	1.00×
	avxbls	$(2 \times 2 \times 2 \times 1)$ -way	2	871	436	2.19×
	avxbls	$(1 \times 2 \times 2 \times 2)$ -way	1	618	618	1.55×
$\mathbb{F}_{p^6}$ multiplication (without reduction)	blst	1-way	1	2046	2046	1.00×
	avxbls	$(2 \times 2 \times 2 \times 1)$ -way	2	1616	808	2.53×
	avxbls	$(1 \times 2 \times 2 \times 2)$ -way	1	1195	1195	1.71×

Per Table 2, each listed arithmetic operation in $\mathbb{F}_{p^2}$ has both a $(4 \times 2 \times 1)$ -way and a $(2 \times 2 \times 2)$ -way implementation. Taking multiplication in $\mathbb{F}_{p^2}$ (without reduction) as an example, the former (resp. the latter) is 3.03 times (resp. 2.23 times) faster than the scalar implementation in **blst**. Notably, these two $\mathbb{F}_{p^2}$ multiplications serve as the core sub-routine in, respectively, the $(2 \times 2 \times 2 \times 1)$ -way and $(1 \times 2 \times 2 \times 2)$ -way multiplication in $\mathbb{F}_{p^6}$. However, the $\mathbb{F}_{p^6}$ multiplication has a smaller speed-up factor, namely only 2.53 (resp. 1.71), which implies that the vector transformation entails a performance penalty.

Table 3 shows that the original pairing implementation in **blst** (2,351,615 cycles) is slower than Longa’s implementation (2,206,902 cycles), mainly because the latter adopts the $\mathbb{F}_{p^{12}}$ compressed cyclotomic squaring. After applying the same technique, our scalar implementation, which uses the $\mathbb{F}_{p^{12}}$ compressed cyclotomic squaring in **blst**, is slightly faster (2,169,338 cycles). Per Table 3, the target $\mathbb{F}_{p^{12}}$ and line operations in **avxbls** are accelerated by factors between 1.69 and 2.26, and these speed-ups propagate well up to the higher layers. The Miller loop becomes 2.01 times faster and the final exponentiation achieves a 1.77-fold (resp. 1.94-fold) speed-up when using the Granger-Scott cyclotomic squaring (resp. the compressed cyclotomic squaring). Finally, our **avxbls** outperforms the baseline implementation by a factor of 1.86 (resp. 1.97) for the full pairing.

Comparison: different operations in **avxbls.** Focusing on Table 3, when looking at the performance gains of the **line_db1** and **line_add** operation, the latter (1.69) is less than the former (1.93). This can be explained by the fact that **line_add** is 2-way vectorized

Table 3: Benchmarks of the high-level arithmetic operations in **blst** (x64 assembly implementation), Longa’s implementation [Lon23] (x64 assembly implementation), and **avxbls** (AVX-512 implementation) on an Intel Core i3-1005G1 CPU. The associated vectorization strategies are summarized in Table 1. The “Granger-Scott” and “compressed” denote the implementation using the Granger-Scott (i.e., the uncompressed) and the compressed version of $\mathbb{F}_{p^{12}}$ cyclotomic squaring, respectively.

	blst	[Lon23]	avxbls
$\mathbb{F}_{p^{12}}$ cyclotomic squaring	3313 (1.00 $\times$)	3450 (0.96 $\times$)	1789 (1.85 $\times$)
$\mathbb{F}_{p^{12}}$ compressed cyclotomic squaring	2346 [†] (1.00 $\times$)	2379 (0.99 $\times$)	1088 (2.16 $\times$)
$\mathbb{F}_{p^{12}}$ multiplication	7653 (1.00 $\times$)	8038 (0.95 $\times$)	4074 (1.88 $\times$)
$\mathbb{F}_{p^{12}}$ sparse multiplication	5057 (1.00 $\times$)	5968 (0.85 $\times$)	2373 (2.13 $\times$)
$\mathbb{F}_{p^{12}}$ squaring	5456 (1.00 $\times$)	5834 (0.94 $\times$)	2412 (2.26 $\times$)
line_dbl	4127 (1.00 $\times$)	4314 (0.96 $\times$)	2141 (1.93 $\times$)
line_add	5293 (1.00 $\times$)	5869 (0.90 $\times$)	3125 (1.69 $\times$)
Miller loop	1,003,400 (1.00 $\times$)	1,071,655 (0.94 $\times$)	499,393 (2.01 $\times$)
Final exponentiation	(Granger-Scott) 1,349,003 (1.00 $\times$)	1,150,152 (1.17 $\times$)	762,186 (1.77 $\times$)
	(compressed) 1,169,728 [†] (1.15 $\times$)		694,729 (1.94 $\times$)
Optimal ate pairing	(Granger-Scott) 2,351,615 (1.00 $\times$)	2,206,902 (1.07 $\times$)	1,265,314 (1.86 $\times$)
	(compressed) 2,169,338 [†] (1.08 $\times$)		1,195,236 (1.97 $\times$)

[†] Our scalar implementation that uses the $\mathbb{F}_{p^{12}}$ compressed cyclotomic squaring in **blst**.

at the $\mathbb{F}_{p^2}$ -level and its overall vectorization is $(1 \times 2 \times 2 \times 2)$ -way, whereas **line_dbl** is vectorized in 4-way at the $\mathbb{F}_{p^2}$ -level and overall in $(1 \times 4 \times 2 \times 1)$ -way. Apparently, the $\mathbb{F}_p$ implementations are different, and, as expected by Observation 1, the one based on (8×1) -way $\mathbb{F}_p$ implementation, i.e., **line_dbl**, is accelerated more significantly.

The analysis for $\mathbb{F}_{p^{12}}$ operations is more complicated. First of all, it is clear that the $\mathbb{F}_{p^{12}}$ squaring should have the largest speed-up factor (2.26) because it is 1) implemented with Karatsuba’s method, equivalently to its scalar correspondence, and 2) vectorized in a $(1 \times 4 \times 2 \times 1)$ -way, i.e., using the (8×1) -way $\mathbb{F}_p$ implementation. The sparse multiplication in $\mathbb{F}_{p^{12}}$ is based on the schoolbook technique instead of Karatsuba since the latter saves only three $\mathbb{F}_{p^2}$ multiplications, i.e., the impact on efficiency is low. Its main operation, the combined $\mathbb{F}_{p^6}$ sparse multiplication, also uses (8×1) -way $\mathbb{F}_p$ arithmetic and contributes to a similarly significant speed-up factor of 2.13. On the other hand, the cyclotomic squaring in $\mathbb{F}_{p^{12}}$ is accelerated by a smaller factor of around 1.85. It employs a hybrid vectorization to compute the three $\mathbb{F}_{p^4}$ coefficients, whereby the computation of coefficient A , as indicated at line 8-12 in Figure 2, is in $(1 \times 2 \times 2 \times 2)$ -way, which is based on a less-efficient $\mathbb{F}_p$ implementation, i.e., (4×2) -way. In contrast, the compressed version of the $\mathbb{F}_{p^{12}}$ cyclotomic squaring computes only the two $\mathbb{F}_{p^4}$ coefficients B and C , completely realized in $(2 \times 2 \times 2 \times 1)$ -way, so that its speed-up factor is relatively high (roughly 2.16). Finally, the $\mathbb{F}_{p^{12}}$ multiplication is a similar case as the $\mathbb{F}_{p^{12}}$ cyclotomic squaring, where hybrid vectorization is used and one $\mathbb{F}_{p^6}$ multiplication is implemented in $(1 \times 2 \times 2 \times 2)$ -way, which results in a relatively low acceleration factor (1.88).

Comparison: avxbls and avxsike. As mentioned earlier, there already exist a couple of AVX-512IFMA implementations of isogeny-based key-establishment schemes, such as [KG19, CFG⁺21, CFGR22]. The SIKE implementation in [CFGR22], whose associated library is **avxsike**, 1) also uses only (8×1) -way and (4×2) -way $\mathbb{F}_p$ implementations at the bottom layer and 2) achieves a significant performance gain. We aim to compare **avxbls** with **avxsike** on the achieved speed-up factors and analyze how the structure of the primitive affects the vectorization. The timings of **avxsike** from [CFGR22] were

measured on the same CPU (Intel Core i3-1005G1), and the baseline implementation is also written in x64 assembly, meaning a direct comparison is possible. Notably, compared to `avxbls`, `avxsi` reaches higher speed-up factors, namely 3.21 for encapsulation and 2.45 for decapsulation of SIKEp503. The reasons are as follows: 1) SIKE performs the point and isogeny operations via $\mathbb{F}_{p^2}$ arithmetic directly, while our target pairing uses an $\mathbb{F}_{p^{12}}$ tower extension, i.e., more field reductions are needed, which is unfriendly towards vectorization; 2) SIKE allows better parallelization at higher layers. For example, in the 3^e -isogeny computation and evaluation, many 3-isogeny evaluations (at different points) can be carried out in parallel, enabling an up to $(8 \times 1 \times 1 \times 1)$ -way vectorization. In `avxbls`, on the other hand, an 8-way parallelization is only possible at the $\mathbb{F}_p$ layer.

Analysis of vector-scalar hybrid implementation. The vectorized field-multiplications of `avxbls` execute `vpmadd52luq` and `vpmadd52huq` instructions (with 512-bit vectors as operands and result), which are dispatched to port 0. Other ports for arithmetic/logical instructions, such as port 1 and port 5, are relatively idle. The fact that scalar multiply instructions (e.g., `mulx`) are dispatched to port 1 and port 5 naturally raises the question whether a hybrid approach utilizing both vector and scalar multiply instructions can be more efficient¹³. We conducted a number of simple experiments to explore this idea using the multiplication of 381-bit integers, whose execution time is dominated by limb-wise multiplications, as a case study. Considering both (8×1) -way and (4×2) -way parallel implementations as starting point, we found that a hybrid variant that interleaves one vector integer multiplication with one scalar integer multiplication gives the best “cycles per instance” results. Concretely, for the (8×1) -way case, the vector-only multiplication takes 118 cycles (i.e., 14.8 cycles/instance), while the “one vector plus one scalar” hybrid implementation needs 126 cycles (i.e., 14.0 cycles/instance), and the “one vector plus two scalar” hybrid one takes 170 cycles (i.e., 17.0 cycles/instance). As for the (4×2) -way case, the vector-only implementation takes 76 cycles (i.e., 19.0 cycles/instance), while the “one vector plus one scalar” hybrid implementation executes in 90 cycles (i.e., 18.0 cycles/instance), and the “one vector plus two scalar” hybrid variant requires 142 cycles (i.e., 23.7 cycles/instance). However, we note: 1) the “one vector plus one scalar” hybrid method provides 9-way and/or 5-way parallelism, which is difficult to exploit in `avxbls` since, as shown in Section 3, most of the routines are ideally vectorized in 2-, 4-, or 8-way; 2) the performance gain due to this hybrid approach is only marginal; and 3) the scalar part adopts a 64-bit-per-word representation, which means an actual integration into `avxbls` will introduce some overheads (e.g., radix/data transformations), which are not included in the experiments. Taking these overheads into account will further reduce the relatively marginal speed-up or make hybrid approach even slower than the current implementation. As a result, we decided to not use this vector-scalar hybrid technique in `avxbls`.

5 Conclusions

Summary. Some implications of Moore’s Law, in particular the importance of thermal management leading to limitations on clock frequency, have led to an increased focus on performance gains through parallelism. Alongside the prevalence of multi-core CPUs, one clear instance has been expansion of support for the vector (SIMD) programming model offered in associated architectures: such support has increased the level of computational power available in commodity CPUs, *if* one can harness it.

¹³With respect to port congestion, the `vpmadd52luq/vpmadd52huq` instruction has the benefit of fusing a vector multiplication with a vector addition (e.g., `vpaddq`, dispatched to either port 0 or port 5), thereby reducing port pressure.

In this paper, we demonstrated that Intel’s latest vector extension, i.e., AVX-512, can be harnessed to significantly accelerate the OAP on BLS12-381, whereby our presented vectorization strategies can be adapted to other vector extensions. Taking a vector engine with a (physical) 256-bit data-path (i.e., four 64-bit lanes) as an example, for the sub-routines using (4×2) -way $\mathbb{F}_p$ -arithmetic, one can straightforwardly shift them to base on a (4×1) -way version. Then, for the sub-routines using (8×1) -way $\mathbb{F}_p$ -arithmetic, one can easily find an optimal *overall* vectorization methodology based on our vectorization strategies for different layers. On the other hand, the eight parallel lanes seem to be an ideal setup for vectorizing the OAP on BLS12-381, e.g., 16 lanes would already result in many lanes to be idle in some operations.

Future work. The work presented in this paper paves the way for a number of avenues for future research. First, a natural follow-up is to develop a vectorized implementation of the complete BLS signature, which is the primary objective of the `blst` library. This would involve the vectorization of some new components, in particular the hash-to-curve operation, which requires careful parallelization (and vectorization) of additional curve operations. Second, in addition to the parallelism provided by vector extensions, there is a further form of parallelism, which is enabled by multi-processing (also referred to as multi-threading), and previous work like [ALH10, AKMR11] has shown such computing capability to be highly effective for accelerating the Miller loop by adopting a divide-and-conquer strategy. It would be interesting to apply multi-threaded computing to `avxbls` and also other components of the BLS signature, and to research optimal “vectorization plus multi-processing” strategies.

Acknowledgements

This work has been supported in part by the Luxembourg National Research Fund (FNR) via the CORE projects Real-World Implementation and Human-Centered Design of PAKE Technologies – ImPAKT (C21/IS/16221219), Privacy-Preserving Tokenisation of Artworks – PABLO (C21/IS/16326754), and Advanced Cryptography for Finance and Privacy – CryptoFin (C22/IS/17415825).

References

- [AFG24] D.F. Aranha, G. Fotiadis, and A. Guillevic. A short-list of pairing-friendly curves resistant to the special TNFS algorithm at the 192-bit security level. *IACR Communications in Cryptology (CIC)*, 1(3):44, 2024. <https://doi.org/10.62056/angyl86bm>.
- [AGM⁺] D.F. Aranha, C.P.L. Gouvêa, T. Markmann, R.S. Wahby, and K. Liao. RELIC is an Efficient LIBrary for Cryptography. <https://github.com/relic-toolkit/relic>.
- [AKL⁺11] D.F. Aranha, K. Karabina, P. Longa, C.H. Gebotys, and J. López. Faster explicit formulas for computing pairings over ordinary curves. In *Advances in Cryptology (EUROCRYPT)*, LNCS 6632, pages 48–68. Springer-Verlag, 2011. https://doi.org/10.1007/978-3-642-20465-4_5.
- [AKMR11] D.F. Aranha, E. Knapp, A. Menezes, and F. Rodríguez-Henríquez. Parallelizing the Weil and Tate pairings. In *IMA International Conference on Cryptography and Coding (IMACC)*, LNCS 7089, pages 275–295. Springer-Verlag, 2011. https://doi.org/10.1007/978-3-642-25516-8_17.

- [ALH10] D.F. Aranha, J. López, and D. Hankerson. High-speed parallel software implementation of the η_T pairing. In *The Cryptographers' Track at the RSA Conference (CT-RSA)*, LNCS 5985, pages 89–105. Springer-Verlag, 2010. https://doi.org/10.1007/978-3-642-11925-5_7.
- [AR19] A. Abel and J. Reineke. uops.info: Characterizing latency, throughput, and port usage of instructions on Intel microarchitectures. In *Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 673–686. ACM, 2019. <https://doi.org/10.1145/3297858.3304062>.
- [BB04] D. Boneh and H. Boyen. Short signatures without random oracles. In *Advances in Cryptology (EUROCRYPT)*, LNCS 3027, pages 56–73. Springer-Verlag, 2004. https://doi.org/10.1007/978-3-540-24676-3_4.
- [BCL08] E. Brickell, L. Chen, and J. Li. A new direct anonymous attestation scheme from bilinear maps. In *Trusted Computing - Challenges and Applications (Trust)*, LNCS 4968, pages 166–178. Springer-Verlag, 2008. https://doi.org/10.1007/978-3-540-68979-9_13.
- [BD19] R. Barbulescu and S. Duquesne. Updating key size estimations for pairings. *Journal of Cryptology (JOC)*, 32(4):1298–1336, 2019. <https://doi.org/10.1007/s00145-018-9280-5>.
- [BF01] D. Boneh and M. Franklin. Identity-based encryption from the Weil pairing. In *Advances in Cryptology (CRYPTO)*, LNCS 2139, pages 213–229. Springer-Verlag, 2001. https://doi.org/10.1007/3-540-44647-8_13.
- [BKD⁺12] A.K. Bhattacharya, S. Karati, A. Das, D.R. Chowdhury, B. Bellur, and A. Iyer. Use of SIMD features to speed up eta pairing. In *E-Business and Telecommunications - International Joint Conference (ICETE)*, CCIS 455, pages 137–154. Springer-Verlag, 2012. https://doi.org/10.1007/978-3-662-44791-8_9.
- [BLS01] D. Boneh, B. Lynn, and H. Shacham. Short signatures from the Weil pairing. In *Advances in Cryptology (ASIACRYPT)*, LNCS 2248, pages 514–532. Springer-Verlag, 2001. https://doi.org/10.1007/3-540-45682-1_30.
- [BLS02] P.S.L.M. Barreto, B. Lynn, and M. Scott. Constructing elliptic curves with prescribed embedding degrees. In *Security in Communication Networks (SCN)*, LNCS 2576, pages 257–267. Springer-Verlag, 2002. https://doi.org/10.1007/3-540-36413-7_19.
- [BN05] P.S.L.M. Barreto and M. Naehrig. Pairing-friendly elliptic curves of prime order. In *Selected Areas in Cryptography (SAC)*, LNCS 3897, pages 319–331. Springer-Verlag, 2005. https://doi.org/10.1007/11693383_22.
- [BN24] D.J. Bernstein and K. Nath. Lib25519: A microlibrary for X25519 and Ed25519, 2024. <https://lib25519.cr.py.to>.
- [Bow17] S. Bowe. BLS12-381: New zk-SNARK elliptic curve construction. Electric Coin Co. Blog Post, 2017. <https://electriccoin.co/blog/new-snark-curve>.
- [BSS05] I.F. Blake, G. Seroussi, and N.P. Smart. *Advances in Elliptic Curve Cryptography*. Cambridge University Press, 2005.

- [CFG⁺21] H. Cheng, G. Fotiadis, J. Großschädl, P.Y.A. Ryan, and P.B. Rønne. Batching CSIDH group actions using AVX-512. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2021(4):618–649, 2021. <https://doi.org/10.46586/tches.v2021.i4.618-649>.
- [CFGR22] H. Cheng, G. Fotiadis, J. Großschädl, and P.Y.A. Ryan. Highly vectorized SIKE for AVX-512. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2022(2):41–68, 2022. <https://doi.org/10.46586/tches.v2022.i2.41-68>.
- [CGT⁺20] H. Cheng, J. Großschädl, J. Tian, P.B. Rønne, and P.Y.A. Ryan. High-throughput elliptic curve cryptography using AVX2 vector instructions. In *Selected Areas in Cryptography (SAC)*, LNCS 12804, pages 698–719. Springer-Verlag, 2020. https://doi.org/10.1007/978-3-030-81652-0_27.
- [Cho15] T. Chou. Sandy2x: New Curve25519 speed records. In *Selected Areas in Cryptography (SAC)*, LNCS 9566, pages 145–160. Springer-Verlag, 2015. https://doi.org/10.1007/978-3-319-31301-6_8.
- [CL04] J. Camenisch and A. Lysyanskaya. Signature schemes and anonymous credentials from bilinear maps. In *Advances in Cryptology (CRYPTO)*, LNCS 3152, pages 56–72. Springer, 2004. https://doi.org/10.1007/978-3-540-28628-8_4.
- [CLN10] C. Costello, T. Lange, and M. Naehrig. Faster pairing computations on curves with high-degree twists. In *Public Key Cryptography (PKC)*, LNCS 6056, pages 224–242. Springer-Verlag, 2010. https://doi.org/10.1007/978-3-642-13013-7_14.
- [CMS08] L. Chen, P. Morrissey, and N.P. Smart. Pairings in trusted computing. In *Pairing-Based Cryptography (Pairing)*, LNCS 5209, pages 1–17. Springer-Verlag, 2008. https://doi.org/10.1007/978-3-540-85538-5_1.
- [Edg23] B. Edgington. Upgrading ethereum: A technical handbook on ethereum’s move to proof of stake and beyond. Edition 0.3: Capella [WIP], 2023. <https://eth2book.info/capella/>.
- [FK19] G. Fotiadis and E. Konstantinou. TNFS resistant families of pairing-friendly elliptic curves. *Journal of Theoretical Computer Science (TCS)*, 800:73–89, 2019. <https://doi.org/10.1016/j.tcs.2019.10.017>.
- [FL15] A. Faz-Hernández and J.C. López-Hernández. Fast implementation of Curve25519 using AVX2. In *Progress in Cryptology (LATINCRYPT)*, LNCS 9230, pages 329–345. Springer-Verlag, 2015. https://doi.org/10.1007/978-3-319-22174-8_18.
- [FLD19] A. Faz-Hernández, J. López, and R. Dahab. High-performance implementation of elliptic curve cryptography using vector instructions. *ACM Transactions on Mathematical Software (TOMS)*, 45(3):1–35, 2019. <https://doi.org/10.1145/3309759>.
- [FST10] D. Freeman, M. Scott, and E. Teske. A taxonomy of pairing-friendly elliptic curves. *Journal of Cryptology (JOC)*, 23(2):224–280, 2010. <https://doi.org/10.1007/s00145-009-9048-z>.

- [GGP08] P. Grabher, J. Großschädl, and D. Page. On software parallel implementation of cryptographic pairings. In *Selected Areas in Cryptography (SAC)*, LNCS 5381, pages 35–50. Springer-Verlag, 2008. https://doi.org/10.1007/978-3-642-04159-4_3.
- [Gro16] J. Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology (EUROCRYPT)*, LNCS 9666, pages 305–326. Springer, 2016. https://doi.org/10.1007/978-3-662-49896-5_11.
- [GS10] R. Granger and M. Scott. Faster squaring in the cyclotomic subgroup of sixth degree extensions. In *Public Key Cryptography (PKC)*, LNCS 6056, pages 209–223. Springer-Verlag, 2010. https://doi.org/10.1007/978-3-642-13013-7_13.
- [GS21] A. Guillevic and S. Singh. On the alpha value of polynomials in the tower number field sieve algorithm. *Mathematical Cryptology*, 1(1):1–39, 2021. <https://journals.flvc.org/mathcryptology/article/view/125142>.
- [Gui20] A. Guillevic. A short-list of pairing-friendly curves resistant to special TNFS at the 128-bit security level. In *Public-Key Cryptography (PKC)*, LNCS 12111, pages 535–564. Springer-Verlag, 2020. https://doi.org/10.1007/978-3-030-45388-6_19.
- [HEY20] H. Hisil, B. Egrice, and M. Yassi. Fast 4 way vectorized ladder for the complete set of Montgomery curves. Cryptology ePrint Archive, Paper 2020/388, 2020. <https://eprint.iacr.org/2020/388>.
- [HHT20] D. Hayashida, K. Hayasaka, and T. Teruya. Efficient final exponentiation via cyclotomic structure for pairings over families of elliptic curves. Cryptology ePrint Archive, Paper 2020/875, 2020. <https://eprint.iacr.org/2020/875>.
- [HSV06] F. Hess, N.P. Smart, and F. Vercauteren. The eta pairing revisited. *IEEE Transactions on Information Theory (TIT)*, 52(10):4595–4602, 2006. <https://doi.org/10.1109/TIT.2006.881709>.
- [Int25a] The converged vector ISA: Intel advanced vector extensions 10 (revision 3). Technical report, Intel Corp., 2025. <https://www.intel.com/content/www/us/en/content-details/849709/the-converged-vector-isa-intel-advanced-vector-extensions-10-technical-paper.html>.
- [Int25b] Intel 64 and IA-32 architectures software developer’s manual, combined volumes: 1, 2a, 2b, 2c, 2d, 3a, 3b, 3c, 3d, and 4. Technical report, Intel Corp., 2025. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>.
- [Jou04] A. Joux. A one round protocol for tripartite Diffie-Hellman. *Journal of Cryptology (JOC)*, 17(4):263–276, 2004. <https://doi.org/10.1007/s00145-004-0312-y>.
- [KB16] T. Kim and R. Barbulescu. Extended tower number field sieve: A new complexity for the medium prime case. In *Advances in Cryptology (CRYPTO)*, LNCS 9814, pages 543–571. Springer-Verlag, 2016. https://doi.org/10.1007/978-3-662-53018-4_20.
- [KG19] D. Kostic and S. Gueron. Using the new VPMADD instructions for the new post quantum key encapsulation mechanism SIKE. In *Computer Arithmetic (ARITH)*, pages 215–218. IEEE, 2019. <https://doi.org/10.1109/ARITH.2019.00050>.

- [KZG10] A. Kate, G.M. Zaverucha, and I. Goldberg. Constant-size commitments to polynomials and their applications. In *Advances in Cryptology (ASIACRYPT)*, LNCS 6477, pages 177–194. Springer, 2010. https://doi.org/10.1007/978-3-642-17373-8_11.
- [Lon23] P. Longa. Efficient algorithms for large prime characteristic fields and their application to bilinear pairings. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2023(3):445–472, 2023. <https://doi.org/10.46586/tches.v2023.i3.445-472>.
- [MAF20] N.B. Mbang, D.F. Aranha, and E. Fouotsa. Computing the optimal ate pairing over elliptic curves with embedding degrees 54 and 48 at the 256-bit security level. *International Journal of Applied Cryptography (IJACT)*, 4(1):45–59, 2020. <https://doi.org/10.1504/IJACT.2020.107167>.
- [Mil04] V.S. Miller. The Weil pairing, and its efficient calculation. *Journal of Cryptology (JOC)*, 17(4):235–261, 2004. <https://doi.org/10.1007/s00145-004-0315-8>.
- [NS21] K. Nath and P. Sarkar. Efficient 4-way vectorizations of the Montgomery ladder. *IEEE Transactions on Computers (TOC)*, 71(3):712–723, 2021. <https://doi.org/10.1109/TC.2021.3060505>.
- [PS16] D. Pointcheval and O. Sanders. Short randomizable signatures. In *The Cryptographers’ Track at the RSA Conference (CT-RSA)*, LNCS 9610, pages 111–126. Springer-Verlag, 2016. https://doi.org/10.1007/978-3-319-29485-8_7.
- [TZ23] S. Tessaro and C. Zhu. Revisiting BBS signatures. In *Advances in Cryptology (EUROCRYPT)*, LNCS 14008, pages 691–721. Springer-Verlag, 2023. https://doi.org/10.1007/978-3-031-30589-4_24.
- [Ver10] F. Vercauteren. Optimal pairings. *IEEE Transactions on Information Theory (TIT)*, 56(1):455–461, 2010. <https://doi.org/10.1109/TIT.2009.2034881>.
- [WP06] A. Weimerskirch and C. Paar. Generalizations of the Karatsuba algorithm for efficient implementations. Cryptology ePrint Archive, Paper 2006/224, 2006. <https://eprint.iacr.org/2006/224>.
- [ZHZ⁺24] J. Zhang, J. Huang, L. Zhao, D. Chen, and Ç.K. Koç. ENG25519: faster TLS 1.3 handshake using optimized X25519 and Ed25519. In *USENIX Security Symposium*, 2024. <https://www.usenix.org/conference/usenixsecurity24/presentation/zhang-jipeng>.